MONTH 1 - JAVA FUNDAMENTALS & DEEP DIVE

1. Java Fundamentals

1.1 Java Introduction

- Java is a high-level, object-oriented, class-based programming language.
- It is platform-independent because Java source code is compiled into bytecode, which can execute on any operating system with a compatible JVM.
- Java follows the principle: **Write Once, Run Anywhere (WORA)**.
- Java supports object-oriented programming concepts such as encapsulation, inheritance, polymorphism and abstraction.
- Java is commonly used for backend development, enterprise applications, distributed systems and Android development.

1.2 Java Development Environment Setup

Set up the Java development environment on Windows.

- **JDK 21:** Installed Java Development Kit to compile and run Java applications.
- **JVM:** Understand its role in executing Java bytecode and managing runtime memory.
- **Maven:** Installed and configured Maven for project management, dependency management and builds.
- **Git:** Installed and configured Git for version control.
- **VS Code:** Installed the Java Extension Pack for Java development, execution and debugging.
- **IntelliJ IDEA:** Studied its role as an alternative Java IDE for writing, running and debugging Java applications.

Project setup:

- Created a Maven-based Java project.
- Understood the Maven project structure, including `src/main/java`, `src/test/java` and `pom.xml`.
- Created the GitHub repository for QueueDesk.
- Wrote and executed the first Java program: `Hello, QueueDesk!`.

Understood **why Java is platform-independent**, how Java code is compiled into bytecode and executed by the JVM.

Platform - consists of processor and OS, c/c++ -> compiler , python -> interpreter and java mai both interpreter and compiler.

1.3 Understanding JDK, JRE and JVM

JVM (Java Virtual Machine)

- Executes compiled Java bytecode (`.class` files).
- Provides **platform independence** by running bytecode on different operating systems.
- Manages runtime memory, including Stack and Heap.
- Performs automatic garbage collection.
- Uses an interpreter and Just-In-Time (JIT) compiler for execution and performance optimization.
- Interpreter supports Execution & JIT compiler - boosts performance
- JIT COMPILER -> Inside the JVM , Hot Code paths(frequently executed bytecode) are compiled into native Machine Code at runtime, significantly boosting performance.

JRE (Java Runtime Environment) - JVM + Core java class libraries(eg : java.util, java.io)

- Provides the environment required to run Java applications.

JDK (Java Development Kit) - JRE + development tools

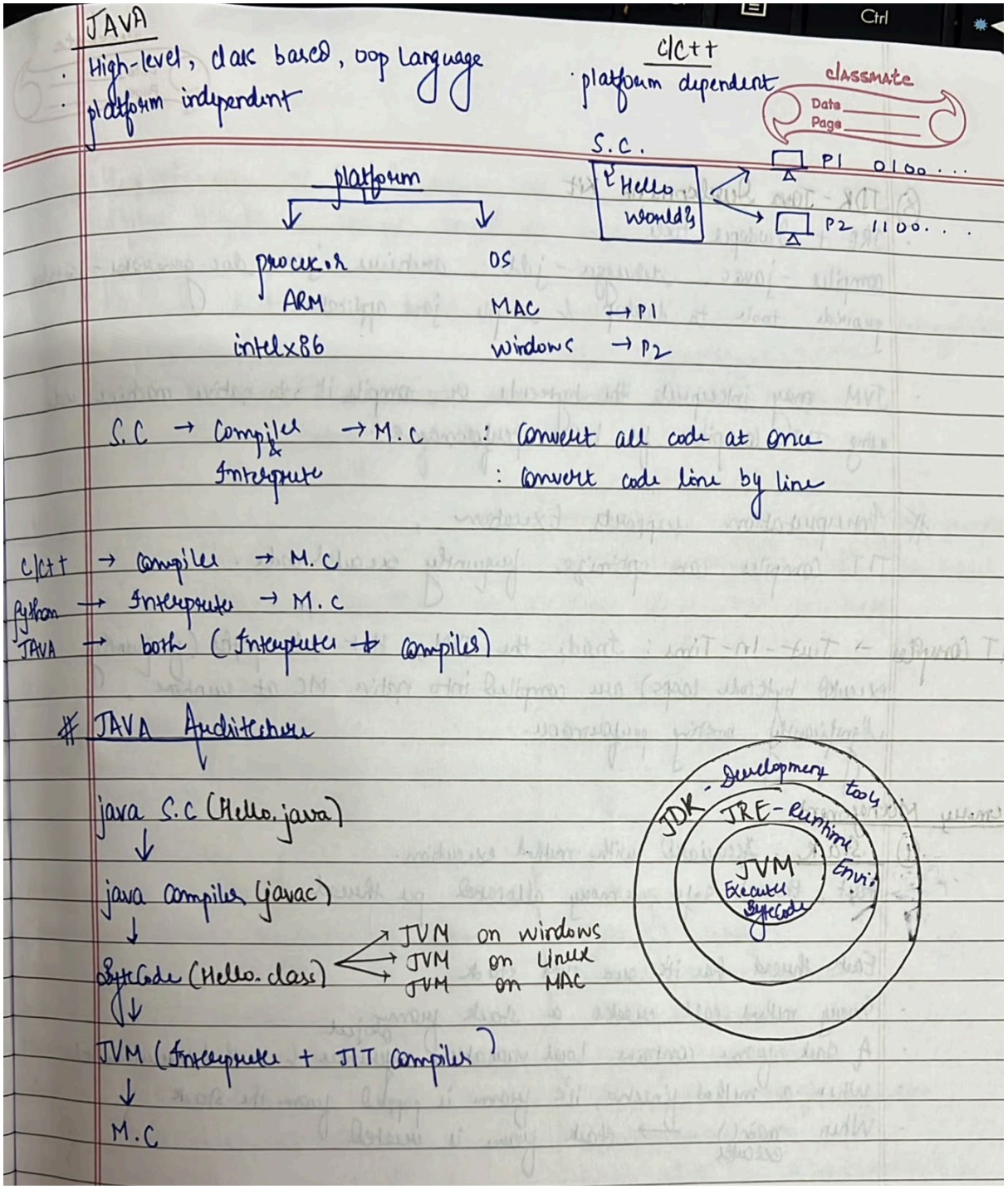
- Provides tools required to develop and compile Java applications.
- Includes the Java compiler (`javac`), debugger(`jdb`) and other development tools, along with the runtime components.

Relationship: JDK → JRE → JVM (conceptual relationship).

1.4 Java Compilation and Execution

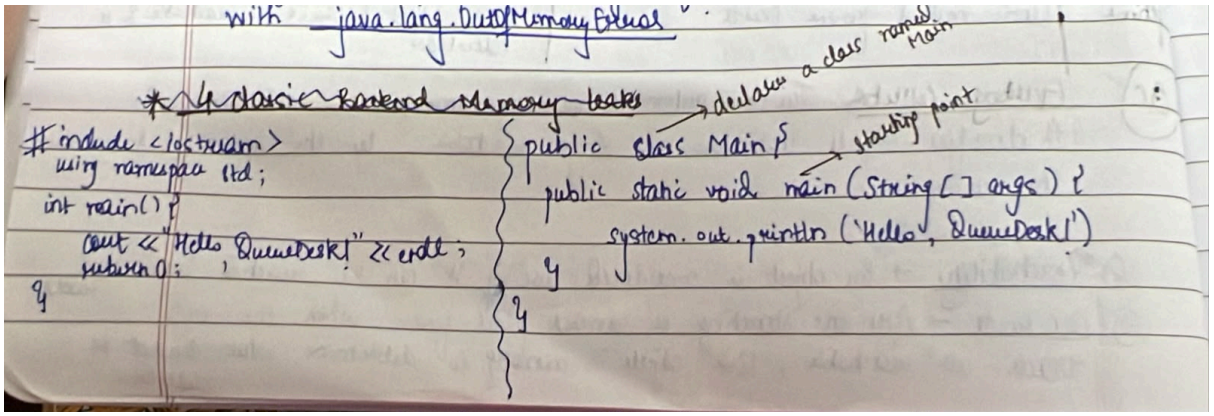
- Java source code is written in `.java` files.
- The Java compiler (`javac`) converts source code into bytecode.
- Bytecode is stored in `.class` files.
- The JVM loads and executes the bytecode.
- The JVM can interpret bytecode and use JIT compilation to compile frequently executed code into native machine code.

Java Architecture



1.5 Java Program Structure and main()

The `main()` method is the standard entry point of a Java application.



Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, QueueDesk!");
    }
}
```

Understanding the components:

- `public`: Makes the method accessible to the Java launcher.
- `static`: Allows the method to be called without creating an object of the Main class.
- `void`: Indicates that the method does not return a value.
- `main`: The name of the method, entry point of java program.The JVM looks for this exact name to start the program. Standard entry-point method name.
- `String[] args`: Accepts command-line arguments. The parameters passed to the method.
- `System.out.println()`: Prints output to the console.

1.6 Variables and Data Types

Variables

- A variable is a named storage location used to hold a value.
- Variables have a data type that determines the kind of value they can store.
- Java supports local, instance and static variables.

Types of variables:

- **Local variables:** Declared inside methods, constructors or blocks.
- **Instance variables:** Belong to individual objects.
- **Static variables:** Belong to the class and are shared across its instances.

Data Types

Java has two main categories of data types:

A. Primitive Data Types

Java provides eight primitive data types:

Data type	Size	Purpose
`byte`	8 bits	Small integer values
`short`	16 bits	Short integer values
`int`	32 bits	Integer values
`long`	64 bits	Large integer values
`float`	32 bits	Single-precision decimals
`double`	64 bits	Double-precision decimals
`char`	16 bits	UTF-16 code unit
`boolean`	JVM-dependent representation	True or false

- Stored directly by value in stack memory.
  Primitive variables hold primitive values directly.
- Primitive types cannot be assigned `null`.

B. Reference Data Types

Examples:

- String
- Arrays
- Classes and objects
- Interfaces
- Enums
- Collections such as List, Set and Map

Key points:

- Reference variables hold references to heap objects.
- Objects and arrays are generally allocated on the heap.
- Reference variables can hold `null`.

1.7 Operators in Java

Operators perform operations on variables and values.

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Relational:** `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logical:** `&&`, `||`, `!`
- **Assignment:** `=`, `+=`, `-=`, `*=`
- **Unary:** `++`, `--`, `!`
- **Ternary:** `condition ? value1 : value2`

The modulus operator (`%`) returns the remainder after division. It is useful for checking divisibility and identifying patterns.

Example:

```java
if (ticketNumber % 5 == 0) {
    System.out.println("VIP Lane");
}
```

1.8 Control Flow Statements

Control flow statements determine the order in which instructions execute.

Conditional statements:

- `if`: Executes a block when a condition is true.
- `if-else`: Selects between two execution paths.
- `else-if`: Checks multiple conditions.
- `switch`: Selects a block based on a matching value.

Loops:

- `for`: Repeats a block for a specified number of iterations.
- `while`: Repeats while a condition remains true.

- `do-while`: Executes at least once before checking the condition.

Loop control:

- `break`: Terminates the loop.
- `continue`: Skips the current iteration and proceeds to the next.

1.9 Input and Output in Java

- Used `Scanner` to accept user input through the terminal.
- `System.out.print()` displays output without moving to a new line.
- `System.out.println()` displays output followed by a new line.
- `Scanner.nextInt()` reads integer input.
- `Scanner.hasNextInt()` checks whether the next input token is a valid integer.

1.10 Java Input Validation

- Validated user input before performing operations.
- Used `hasNextInt()` to prevent invalid non-integer input.
- Checked for negative customer counts.
- Used conditional statements to handle invalid input and terminate execution safely

1.11 Memory Management in Java

Java uses automatic memory management through the JVM. Runtime memory is primarily managed using Stack and Heap memory.

A. Stack Memory

- Associated with method execution.
- Fast, thread-specific memory allocated per thread.
- Each thread has its own JVM stack.
- Every method call creates a stack frame.
- A stack frame contains local variables, object references and method execution information.
- When a method finishes, its stack frame is removed from the stack.
- When `main()` executes, its stack frame is created.
- Deep or infinite recursion can lead to `StackOverflowError`.

Stores:

- Primitive local variables (`int`, `double`, etc.).
- Object references (references to heap objects).

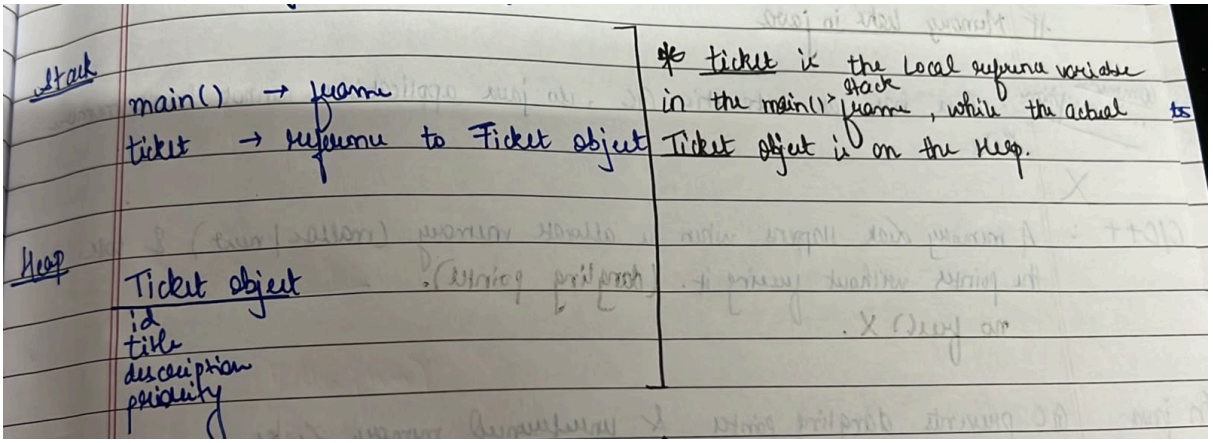
- Method execution frames.

Memory is allocated and released automatically as method frames are pushed and popped.

B. Heap Memory

- Shared memory area accessible across application threads.
- Stores objects and arrays created at runtime.
- All Java objects reside here conceptually (e.g., `new BugTicket(...)`, `new ArrayList<>()`, `new ObjectMapper()`).
- Objects are generally created using the `new` keyword.
- Heap memory is managed by the Garbage Collector (GC).
- Objects remain in the heap until they become unreachable and are eventually collected.

Example:




```
None
public static void main(String[] args) {

    Ticket ticket = new Ticket();

}
```

- `main()` → stack frame.
- `ticket` → local reference variable in the stack frame.
- `new Ticket()` → actual Ticket object in the heap.

C. Stack vs Heap

Feature	Stack	Heap
Stores	Local variables, references, method frames	Objects and arrays
Scope	Per thread	Shared across threads
Allocation	Method execution	Object allocation
Management	Automatic frame lifecycle	Garbage Collector
Access	Generally faster	Objects accessed through references
Lifetime	Until method frame exits	Until object becomes unreachable and is collected
Error	StackOverflowError	OutOfMemoryError

D. Garbage Collection (GC)

- Garbage Collector is Java's automatic memory-management mechanism.
- It identifies objects that are no longer reachable by the application and reclaims their memory.
- Helps prevent manual memory deallocation and dangling-pointer problems.
- Java developers do not explicitly free individual objects.

1. Reachability

- An object is considered live if it can be reached from GC Roots through references.

2. GC Roots

- Starting or anchor points from which the GC traces reachable objects.
- Used to determine which objects are live and which can be collected.

Examples of GC Roots:

- Active local variables and references in stack frames.
- Static fields.
- Active threads.
- JNI (Java Native Interface) references — references to Java objects used by native C/C++ code.

3. Marking

- Identifies reachable objects.

4. Sweeping

- Reclaims memory occupied by unreachable objects.

E. Memory Leak in Java

Common misconception: Java has an automatic Garbage Collector, so Java applications cannot have memory leaks. This is incorrect.

C/C++:

- A memory leak can happen when memory is allocated using `malloc()` or `new`, but the pointer is lost without freeing the memory.
- Example: forgetting `free()` in C or `delete` in C++.

In Java:

- GC prevents manual deallocation errors and dangling-pointer problems.
- However, it cannot prevent logical memory leaks.

Definition:

A memory leak in Java occurs when an object is no longer needed by the business application, but it remains transitively reachable from a GC Root (such as a running thread, static variable or active stack frame).

- Because the object remains reachable, GC cannot collect it.
- Unnecessary objects continue occupying heap memory.
- Heap usage may continuously increase until the JVM experiences `java.lang.OutOfMemoryError`.

F. Classic Backend Memory Leak Scenarios

1. **Unbounded Collections:** ArrayList or HashMap grows continuously without removing unnecessary objects. Solution: bounded storage and cleanup.
2. **ThreadLocal Not Cleared:** Reused server threads retain old request data. Solution: always call `remove()` in a `finally` block.
3. **Cache Without Eviction:** Cache grows indefinitely. Solution: maximum size, TTL and eviction policies such as LRU.
4. **Listeners Not Unregistered:** Long-lived event services retain references to unnecessary listeners. Solution: unregister listeners.
5. **Static Collections:** Static collections retain objects for the lifetime of their classloader. Solution: avoid unbounded static collections.
6. **Singleton Bean Holding Data:** Long-lived Spring singleton services retain unnecessary data. Solution: keep services stateless and externalize persistent storage.

G. Memory Management in QueueDesk

1. Unbounded Repository Storage

```
None
private final List<T> tickets = new ArrayList<>();
```

- Repository stores ticket objects in an ArrayList.
- If tickets are continuously added and never removed, heap usage increases.
- Closed or resolved tickets may remain referenced unnecessarily.

Solution: Database persistence, pagination, archival and retention policies.

2. Dual Queuing in PriorityQ

- `offer(ticket)` inserts the same ticket into two priority queues.
- If consumers only poll one queue, the other retains references.
- Objects cannot be collected while those references remain reachable.

Solution: Keep both queues synchronized and remove unnecessary references.

3. Unclosed File Streams

- Unclosed readers and writers can retain OS file descriptors and I/O resources.
- Used try-with-resources in `TicketFileLoader` and `TicketReportGenerator`.

```
None
try (BufferedReader reader =

        Files.newBufferedReader(path)) {

    // Read file

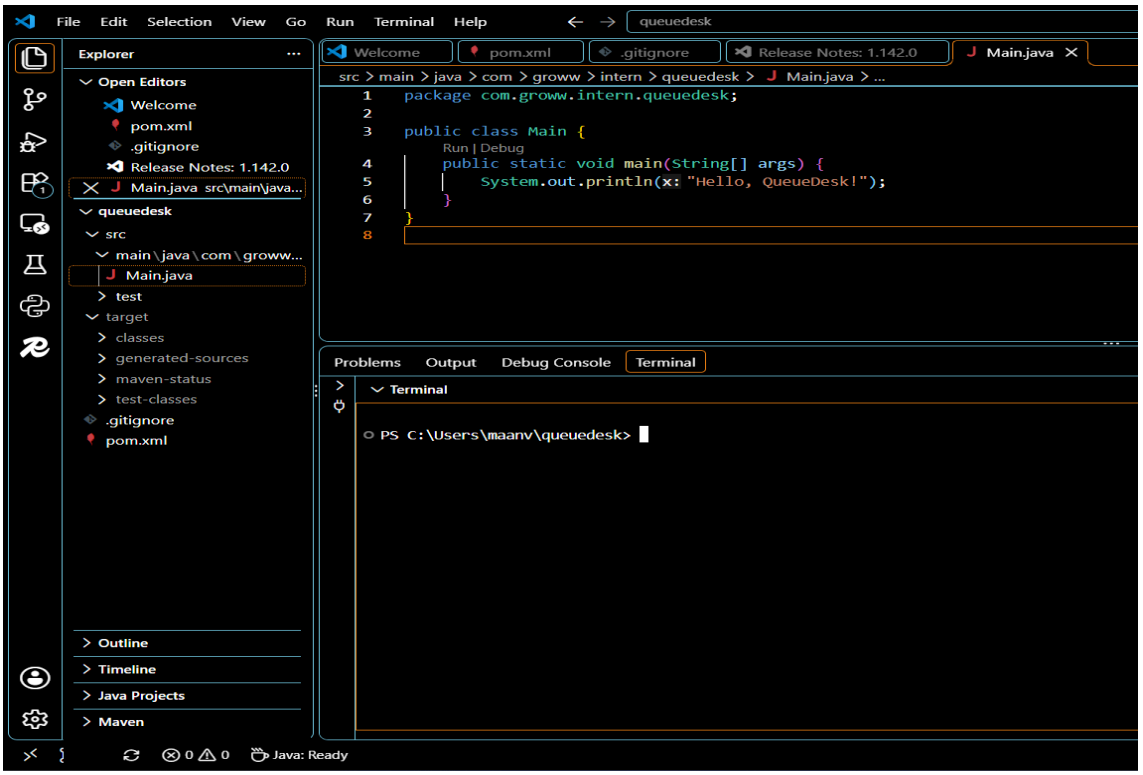
}
```

- Automatically closes resources when the block exits, even if an exception occurs.

Key Takeaway

Java manages memory automatically, but developers are responsible for managing object lifecycles and avoiding unnecessary references. Garbage Collection cannot reclaim objects that remain reachable, even when the application no longer needs them.

JAVA FIRST PROGRAM - “Hello, QueueDesk!”



BUILD-1

- Build: Queue Number Simulator
  - A single file, no classes of your own beyond the entry point.
  - Read a starting ticket number and a count of customers to simulate, via Scanner.
  - Loop through the count, printing "Now serving ticket #N" for each.
  - Flag every 5th number called as a "VIP lane" ticket and every 10th as a "priority recheck", using the modulus operator and if / else (or switch).
  - Track running totals — served, VIP, priority-recheck — using variables and operators only. No arrays or lists yet.

Objective - Build a single-file Java program that simulates serving customers using sequential ticket numbers and tracks VIP-lane and priority-recheck counts.

```
import java.util.Scanner;

public class QueueNumberSimulator {

    public static void main(String[] args){

        Scanner in = new Scanner(System.in); //User input using scanner
, we used scanner to accept the starting ticket number and customer
count from the terminal.

        //accepted from the terminal:starting ticket number and number
of customers to simulate.



        System.out.print("Starting ticket number: ");

        if(!in.hasNextInt()){ //Input validation hasNextInt() checks if
the input is a valid integer. If not, it prompts the user to enter a
whole number and exits the program.

            System.out.println("Please enter a whole number.");

            in.close();//closes the scanner after completing input
processig.

            return;

        }
```

```
        int start = in.nextInt();



        System.out.print("Number of customers to simulate:");

        if(!in.hasNextInt()){

            System.out.println("Please enter a whole number.");

            in.close();

            return;

        }

        int count = in.nextInt();



        if(count < 0){

            System.out.println("Count cannot be negative.");

            in.close();

            return;

        }

        int served = 0, vip = 0,  recheck = 0; //used variables to
maintain total customers served, total vip-lane tickets, total
priority-recheck tickets.

        for(int i = 0 ; i < count ; i++){ //used a for lop to itreate
through the customer count. Generated sequential ticket numbers
starting from the user-provided number.

            int ticket = start + i;

            served++;
```

```
            System.out.println("Now serving ticket #" +
ticket);//printed "Now serving ticket #N" for every customer served.



            //Used the modulus operator (%) and conditional statements
to classify tickets

            if(served % 10 == 0){//Every 10th customer served receives
the priority recheck flag.

                recheck++;

                System.out.println(" -> PRIORITY RECHECK");

            }

            else if(served % 5 == 0){ //Every 5th customer served
receives the VIP-lane flag.

                vip++;

                System.out.println(" -> VIP lane");

            }

        }

        System.out.println("Total served:" + served);

        System.out.println("Total VIP: " + vip);

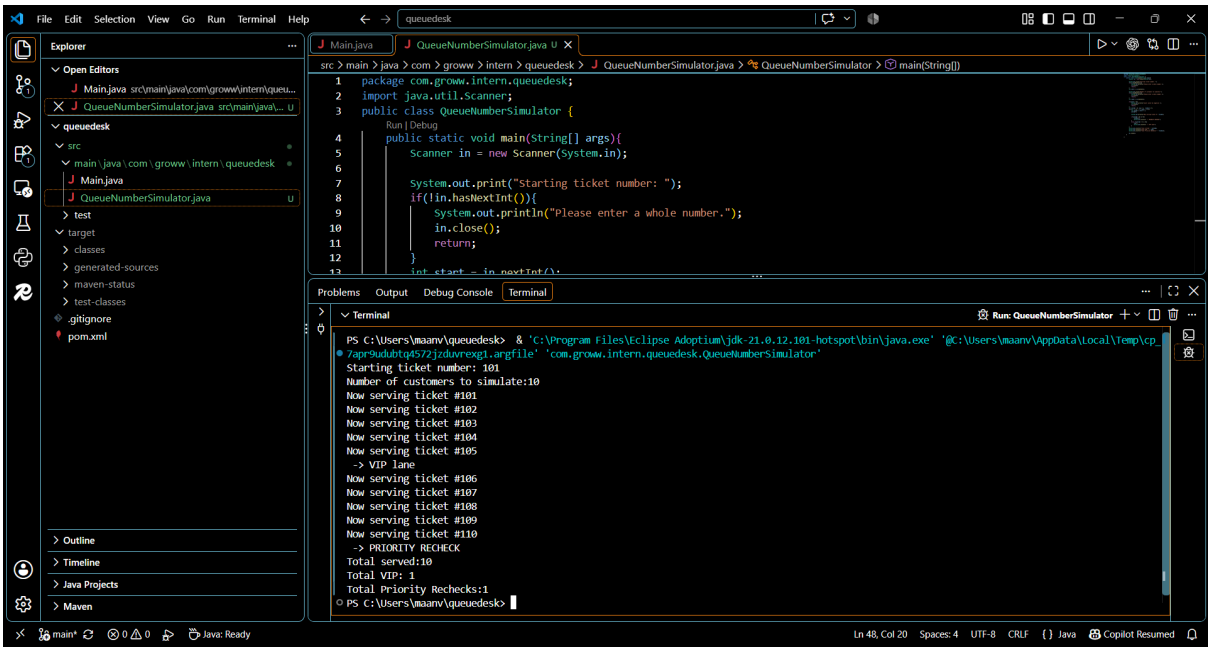
        System.out.println("Total Priority Rechecks:" + recheck);

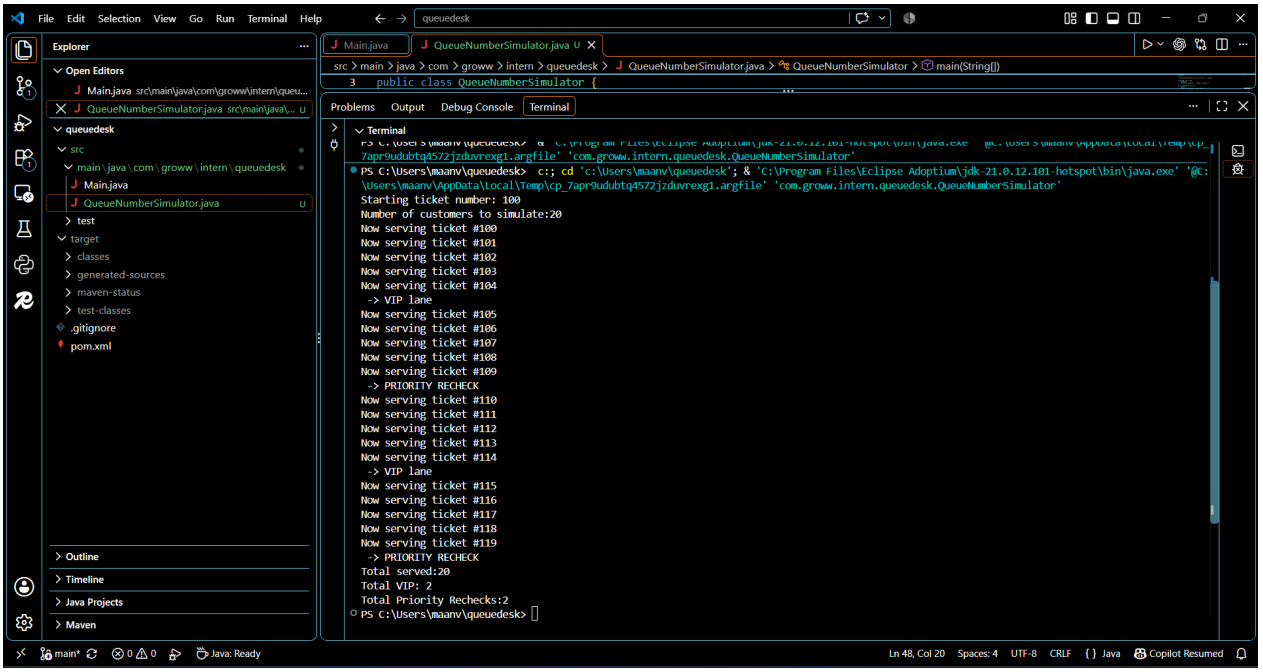


        in.close();

    }
```

QueueNumberSimulator.java





OOP I — Classes, Objects, Constructors and Encapsulation

1. Classes and Objects

- **Class:** A blueprint or template defining properties (fields) and behaviours (methods).
- **Object:** An instance of a class, created at runtime using `new`.
- Objects have state (fields) and behaviour (methods).

```java
class Ticket {
    String title;

    void display() {
        System.out.println(title);
    }
}

// Object creation
Ticket ticket = new Ticket();
ticket.title = "Login Error";
ticket.display();
```

2. Constructors

- A special class member used to initialize objects.
- Has the same name as the class.
- Has no return type, not even `void`.
- Automatically invoked when an object is created using `new`.
- Can be overloaded with different parameter lists.
- Java provides a default constructor only when no constructor is explicitly defined.

```java
class Ticket {
    String title;
```

```
    Ticket(String title) {
        this.title = title;
    }
}
```

3. Encapsulation

- Wrapping data (fields) and methods into a single unit (class).
- Achieved by making fields `private` and providing controlled access through public methods.
- Protects data from direct, uncontrolled modification.
- Improves maintainability and validation.

Java
```
class Ticket {
    private String title;

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        if (title != null && !title.isBlank()) {
            this.title = title;
        }
    }
}
```

4. `this` Keyword

- Refers to the current object.
- Used to distinguish instance variables from local variables or constructor parameters.
- Can call another constructor in the same class using `this()`.
- `this()` must be the first statement in a constructor.

```java
Ticket(String title) {
    this.title = title;
}
```

5. Static vs Instance

Static	Instance
Belongs to the class	Belongs to an object
One shared copy	Separate copy per object
Access using class name	Access using object
Cannot directly access instance members	Can access static and instance members
Created/associated with class initialization	Associated with object creation

```java
class Ticket {
    static int totalTickets = 0;
    String title;

    Ticket(String title) {
        this.title = title;
        totalTickets++;
    }
}
```

- `totalTickets` is shared across all Ticket objects.
- `title` is different for each Ticket object.

OOP II — Inheritance, Polymorphism, Abstract Classes, Interfaces and super

1. Inheritance

- Allows one class to acquire properties and methods of another class.
- Promotes code reuse.
- `extends` keyword is used for class inheritance.
- Java supports single, multilevel and hierarchical class inheritance.
- Java does not support multiple inheritance through classes.

```java
class Ticket {
    void display() {
        System.out.println("Ticket");
    }
}

class BugTicket extends Ticket {
    void reportBug() {
        System.out.println("Bug reported");
    }
}
```

2. Polymorphism

Polymorphism means one interface or method name can represent different behaviours.

A. Compile-time Polymorphism (Method Overloading)

- Same method name with different parameter lists.
- Resolved at compile time.
- Return type alone cannot distinguish overloaded methods.

```java
void add(int a, int b) { }
void add(double a, double b) { }
```

B. Runtime Polymorphism (Method Overriding)

- Child class provides its own implementation of a parent-class method.
- Same method signature.
- Resolved at runtime through dynamic method dispatch.

```java
class Ticket {
    void display() {
        System.out.println("General Ticket");
    }
}

class BugTicket extends Ticket {
    @Override
    void display() {
        System.out.println("Bug Ticket");
    }
}
```

3. Abstract Classes

- Declared using the `abstract` keyword.
- Cannot be instantiated directly.
- Can contain both abstract methods (without a body) and concrete methods (with a body).
- Can have constructors, fields and static methods.
- Child classes must implement inherited abstract methods unless they are also abstract.

```java
abstract class Ticket {
    abstract void calculatePriority();

    void display() {
        System.out.println("Ticket details");
    }
}

class BugTicket extends Ticket {
    @Override
    void calculatePriority() {
        System.out.println("Calculating bug priority");
    }
```

```
}
```

4. Interfaces

- Define a contract that implementing classes agree to follow.
- Declared using `interface`.
- Classes implement interfaces using the `implements` keyword.
- A class can implement multiple interfaces.
- Interfaces can contain abstract methods, default methods, static methods and constants.
- Useful for abstraction and achieving multiple inheritance of type.

```java
interface Notifiable {
    void sendNotification();
}

class EmailNotification implements Notifiable {
    @Override
    public void sendNotification() {
        System.out.println("Email sent");
    }
}
```

5. `super` Keyword

- Refers to the immediate parent class.
- Used to access parent-class variables when hidden by child-class variables.
- Used to call parent-class methods.
- `super()` calls the parent-class constructor.
- `super()` must be the first statement in a constructor.

```java
class Ticket {
    Ticket(String title) {
        System.out.println(title);
```

```
    }
}

class BugTicket extends Ticket {
    BugTicket() {
        super("Login Error");
    }
}
```

6. Abstract Class vs Interface

Abstract Class	Interface
Uses `abstract class`	Uses `interface`
Extended using `extends`	Implemented using `implements`
Can have instance variables	Fields are implicitly `public static final`
Can have constructors	Cannot have constructors
A class extends only one class	A class can implement multiple interfaces
Can contain abstract and concrete methods	Can contain abstract, default, static and private methods

BUILD-2

- Build: the Ticket class

- Fields: id, title, description, requester, priority (enum Priority: LOW, MEDIUM, HIGH, CRITICAL), status (enum Status: OPEN, IN_PROGRESS, RESOLVED, CLOSED), createdAt (LocalDateTime.now()).
- A private static int nextId, incremented in every constructor.
- Two constructors: a full one taking title, description, requester, priority; and a short one taking just title and requester, which chains to the full one via this(...) with sensible defaults.
- Getters for every field. Setters only for description, priority, and status.
- An instance method ageInHours() computing time elapsed since createdAt.
- A static method Ticket.totalCreated() returns how many tickets have ever been made, backed by nextId.

Ticket.java

```
/*
REQ 1: CREATE THE FIELDS
A field is a variable declared inside a class that stores an object's
data.
In Java, these variables inside a class are called 'fields'.
id           -> int            -> Unique ticket ID
title        -> String         -> Short summary of the issue
description  -> String         -> Details of the issue
requester    -> String         -> Person who raised the ticket
priority     -> Priority       -> LOW, MEDIUM, HIGH, CRITICAL
status       -> Status         -> OPEN, IN_PROGRESS, RESOLVED, CLOSED
createdAt    -> LocalDateTime  -> Date and time the ticket was created
Priority.java and Status.java are enum that defines the fixed sets of
allowed values.
enum(short for enumeration) is a Java data type used when a variable
should have one value from a fixed set of predefined options. think of
it like a dropdown menu : u can select one of the available options but
can't select a random value.
Example:
```

```
enum Priority { LOW, MEDIUM, HIGH, CRITICAL }
enum Status { OPEN, IN_PROGRESS, RESOLVED, CLOSED }
So, instead of using a string, we create an enum. : 1.Restrict values
to predefined options, 2.Prevent invalid values and many typing
mistakes, hence -> makes code more reliable.

REQ-2 : GENERATE TICKET IDs AUTOMATICALLY
private static int nextID = 1;

static -> One shared variable for the entire Ticket class.
nextId -> Stores the next ID to assign.
nextId++ -> Assigns the current ID and then increments it.

when creating a ticket, this.id = nextId++; this assigns the current
counter value to the new ticket and then increments the counter.
this.id means the id belonging to the current ticket.
Example: First ticket gets ID 1, second gets ID 2, third gets ID 3.

REQ 3: CONSTRUCTORS
Constructor = special method that initializes an object.
It runs automatically when we use 'new'.

1. Full constructor:
Accepts title, description, requester and priority.
Automatically generates ID, sets status to OPEN, and records the
current creation time.

Ticket a = new Ticket(
    "Login fails",
    "Error 500 on submit",
    "asha",
    Priority.HIGH
);

2. Short constructor:
Takes only title and requester.
Accepts only title and requester.
Defaults: description = "", priority = MEDIUM.

Ticket b = new Ticket("Printer jam", "kiran");
```

this(...) = calls another constructor in the same class. It avoids repeating initialization code.
It must be the first statement in the constructor.

* The full constructor initializes everything; the short constructor supplies defaults and reuses the full constructor through this(...).

REQ 4: private AND final
private -> Prevents direct access to a field from outside the class.
final -> Prevents reassignment after initialization.

id, title, requester and createdAt are final.
description, priority and status can be changed.

REQ 5: GETTERS AND SETTERS
Getter -> Returns/access a field's value.
Setter -> Updates/modifies a field's value.

Getters are provided for all fields.
Setters are provided only for description, priority and status.

REQ 6: AGE OF A TICKET
LocalDateTime.now() -> Gets the current local date and time.
Duration.between() -> Calculates elapsed time.
toHours() -> Returns elapsed whole hours.

ageInHours() returns how many whole hours have passed since creation.

REQ 7: COUNT TOTAL TICKETS
Ticket.totalCreated() -> Returns the total tickets created in this program run.
static -> The method belongs to the class, not an individual ticket.
Implementation: nextId - 1.

TicketsCreated	nextId	nextId - 1
0	1	0
1	2	1
2	3	2
3	4	3

```
REQ 8: VALIDATION
title == null || title.isBlank()
Rejects a missing or blank title.

Objects.requireNonNull()
Rejects null values for required fields.

trim() -> Removes leading and trailing spaces.

REQ 9: toString()
Provides a readable text representation of a ticket.
Ticket
ID - 1
Title - Login Issue
Priority - HIGH
Status - OPEN
Requester - Asha

Your Java object already contains all this information.
What is the problem?
When we write: System.out.println(t1);


You want Java to print the ticket details in a readable format.

So we tell Java how to display the ticket using toString().
public String toString() {
    return "#" + id + " [" + priority + "/" + status
            + "] " + title + " (" + requester + ")";
}

This produces:
#1 [HIGH/OPEN] Login issue (Asha)
*/
package com.groww.intern.queuedesk;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.Objects;
```

```java
public class Ticket {
    private static int nextId = 1; //shared by all tickets

    private final int id;
    private final String title;
    private String description;
    private final String requester;
    private Priority priority;
    private Status status;
    private final LocalDateTime createdAt;

    //full constructor
    /*can be called using Ticket t1 = new Ticket("Login issue", "Error 500", "Asha", Priority.HIGH); */
    public Ticket(String title, String description, String requester, Priority priority){
        this.id = nextId++;
        this.title = Objects.requireNonNull(title, "Title"); //syntax: Objects.requireNonNull(variableToCheck, "Error message if it is null");
        this.description = description == null? "" : description.trim(); //description is optional while title, requester and priority are treated as required fields, this uses a ternary operator which is short of if-else. If the description is null, store an empty string "".Otherwise, store the description after trimming spaces.
        this.requester = Objects.requireNonNull(requester, "Requester").trim();
        this.priority = Objects.requireNonNull(priority, "Priority");
        this.status = Status.OPEN;
        this.createdAt = LocalDateTime.now();

        if(this.title.isBlank()){ //here, we are validating the title if it is empty or contains only spaces, java throws an error instead of accepting the ticket.
            throw new IllegalArgumentException("Title cannot be blank"); //Q:why not use isEmpty()? if we use "" -> return true but if "  " -> return false. isBlank() returns true for both "" and "   ".
        }
    }
```

```
    //short constructor
    /*suppose u want to create a ticket but don't want to provide a
description or priority.
    Ticket t2 = new Ticket("printer issue", "kiran");
    The short constructor supplies the value automatically for the
missing fields. like Description : "" , Priority : Priority.MEDIUM
    this(..) -->It calls another constructor in the same class — in
this case, the full constructor. So, rather than repeating all the
initialization code, Java reuses the full constructor.
    this(...) must be the first statement inside the constructor. */

    public Ticket(String title, String requester){
        this(title, "", requester, Priority.MEDIUM);
    }

    //Getters
    public int getId(){
        return id;
    }
    public String getTitle(){
        return title;
    }
    public String getDescription(){
        return description;
    }
    public Priority getPriority(){
        return priority;
    }
    public Status getStatus(){
        return status;
    }
    public LocalDateTime getCreatedAt(){
        return createdAt;
    }

    //Setters - only for description, priority and status. id, title,
requester and createdAt are final and cannot be changed after
initialization.
    public void setDescription(String description){
```

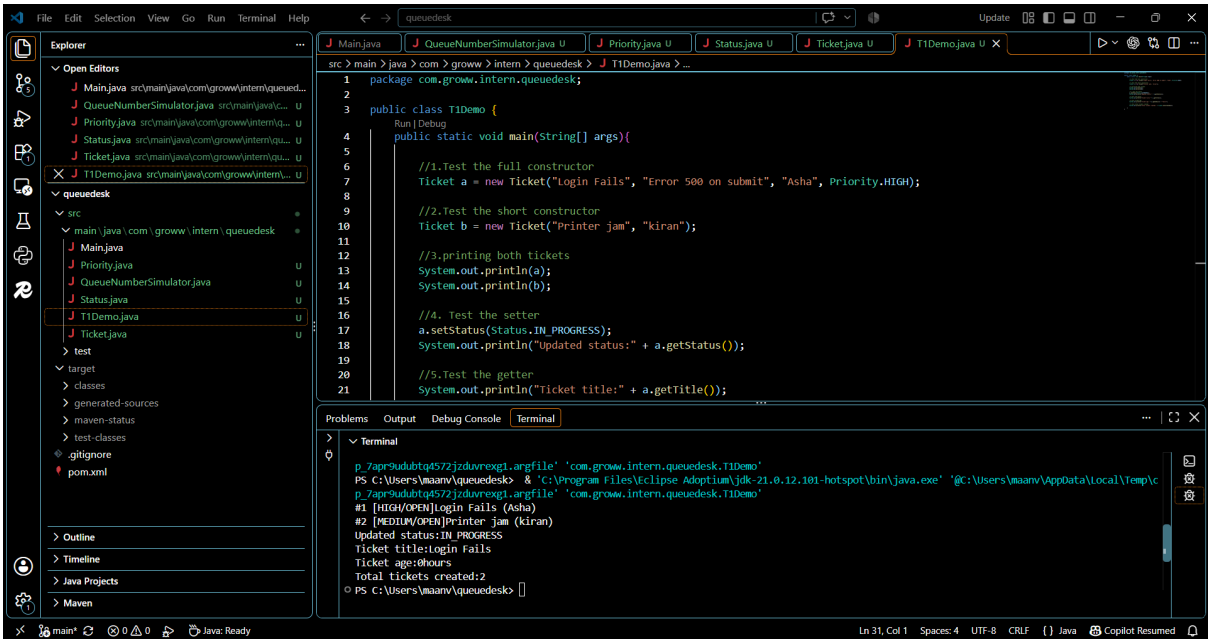
```
        this.description = description == null? "" :
description.trim();
    }
    public void setPriority(Priority priority){
        this.priority = Objects.requireNonNull(priority);
    }
    public void setStatus(Status status){
        this.status = Objects.requireNonNull(status); //eg:
t1.setStatus(Status.IN_PROGRESS);
    }

    //returns the age of the ticket in hours, calculated as the
difference between the current time and the ticket's creation time.
    public long ageInHours(){
        return Duration.between(createdAt,
LocalDateTime.now()).toHours();
    }

    //How many tickets have been created? we can use for eg :
System.out.println(Ticket.totalCreated());
    public static int totalCreated(){
        return nextId - 1; //nextId is incremented after each ticket
creation, so the total number of tickets created is nextId - 1.
    }

    //java already knows how to print an object, but it's default
output isn't useful to us, so we write our own toString() method to
provide a more meaningful representation of the ticket object.
    @Override
    public String toString(){
        return "#" + id + " [" + priority + "/" + status + "]" + title
+ " (" + requester + ")";
    }
}
```

T1.Demo.java


File Edit Selection View Go Run Terminal Help
queuedesk
Update
Explorer
Open Editors
Main.java src\main\java\com\groww\intern\queued...
QueueNumberSimulator.java src\main\java\c... U
Priority.java src\main\java\com\groww\intern\q... U
Status.java src\main\java\com\groww\intern\qu... U
Ticket.java src\main\java\com\groww\intern\qu... U
T1Demo.java src\main\java\com\groww\intern\... U
queuedesk
src
main \ java \ com \ groww \ intern \ queuedesk
Main.java
Priority.java U
QueueNumberSimulator.java U
Status.java U
T1Demo.java U
Ticket.java U
test
target
classes
generated-sources
maven-status
test-classes
.gitignore
pom.xml
Outline
Timeline
Java Projects
Maven
Main.java
QueueNumberSimulator.java U
Priority.java U
Status.java U
Ticket.java U
T1Demo.java U
src > main > java > com > groww > intern > queuedesk > T1Demo.java > ...
1 package com.groww.intern.queuedesk;
2
3 public class T1Demo {
Run | Debug
4 public static void main(String[] args){
5
6 //1.Test the full constructor
7 Ticket a = new Ticket("Login Fails", "Error 500 on submit", "Asha", Priority.HIGH);
8
9 //2.Test the short constructor
10 Ticket b = new Ticket("Printer jam", "kiran");
11
12 //3.printing both tickets
13 System.out.println(a);
14 System.out.println(b);
15
16 //4. Test the setter
17 a.setStatus(Status.IN_PROGRESS);
18 System.out.println("Updated status:" + a.getStatus());
19
20 //5.Test the getter
21 System.out.println("Ticket title:" + a.getTitle());
Problems Output Debug Console Terminal
Terminal
p_7apr9udubtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T1Demo'
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\bin\java.exe' '@C:\Users\maanv\AppData\Local\Temp\c
p_7apr9udubtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T1Demo'
#1 [HIGH/OPEN]Login Fails (Asha)
#2 [MEDIUM/OPEN]Printer jam (kiran)
Updated status:IN_PROGRESS
Ticket title:Login Fails
Ticket age:0hours
Total tickets created:2
PS C:\Users\maanv\queuedesk>
main*
Java: Ready
Ln 31, Col 1
Spaces: 4
UTF-8
CRLF
Java
Copilot Resumed

BUILD-3

- Build: subclasses and a capability
  - Make Ticket abstract; add abstract double estimateEffortHours();.
  - BugTicket extends Ticket — adds severity (enum Severity) and stepsToReproduce.
  - FeatureRequestTicket extends Ticket — adds votes and businessJustification.
  - AccessRequestTicket extends Ticket — adds systemName and accessLevel.
  - Interface Escalatable: void escalate(); boolean isEscalated(); int escalationLevel(); plus one default method, e.g. default String escalationBadge(). Implement it on BugTicket and AccessRequestTicket only.
  - Write printTriageBoard(List<Ticket> tickets) that loops over the base type calling estimateEffortHours() and toString(), then uses if (ticket instanceof Escalatable e) to conditionally show escalation state.

In ticket.java , make these modifications.

```
//Build- T3 Req1: Ticket class is abstract because we don't want to create a generic ticket, we want to create specific types of tickets like BugTicket, FeatureRequestTicket, etc. An abstract class cannot be instantiated directly. It serves as a blueprint for other classes.
public abstract class Ticket
```

```
  //Build-T3 Req2: Add an abstract method to estimate the effort required to resolve the ticket.Each ticket type estimates it's effort differently, so we leave the implementation to the subclasses. This method will be implemented in the subclasses like BugTicket, FeatureRequestTicket, etc.
    public abstract double estimateEffortHours();
```



When we made the Ticket class abstract, errors were encountered in T1Demo.java as the abstract class cannot be instantiated.

```
package com.groww.intern.queuedesk;
//build T3 Req2: Severity enum is used to represent the severity of a
ticket. It can be LOW, MEDIUM, HIGH or CRITICAL. This can be used to
prioritize the tickets and to estimate the effort required to resolve
the ticket.
public enum Severity {
    LOW, MEDIUM, HIGH, CRITICAL
}
```

After making it abstract

```
package com.groww.intern.queuedesk;

public class T2Demo {
    public static void main(String[] args) {

        // 1. Create a bug ticket using the full constructor
        BugTicket a = new BugTicket(
            "Login Fails",
            "Error 500 on submit",
            "Asha",
            Priority.HIGH,
            Severity.CRITICAL,
            "Submit the login form"
        );

        // 2. Create another bug ticket
        BugTicket b = new BugTicket(
            "Printer jam",
            "",
            "Kiran",
            Priority.MEDIUM,
            Severity.LOW,
            "Check the printer paper tray"
        );

        // 3. Print both tickets
```

```
        System.out.println(a);
        System.out.println(b);

        // 4. Test the status setter
        a.setStatus(Status.IN_PROGRESS);
        System.out.println("Updated status: " + a.getStatus());

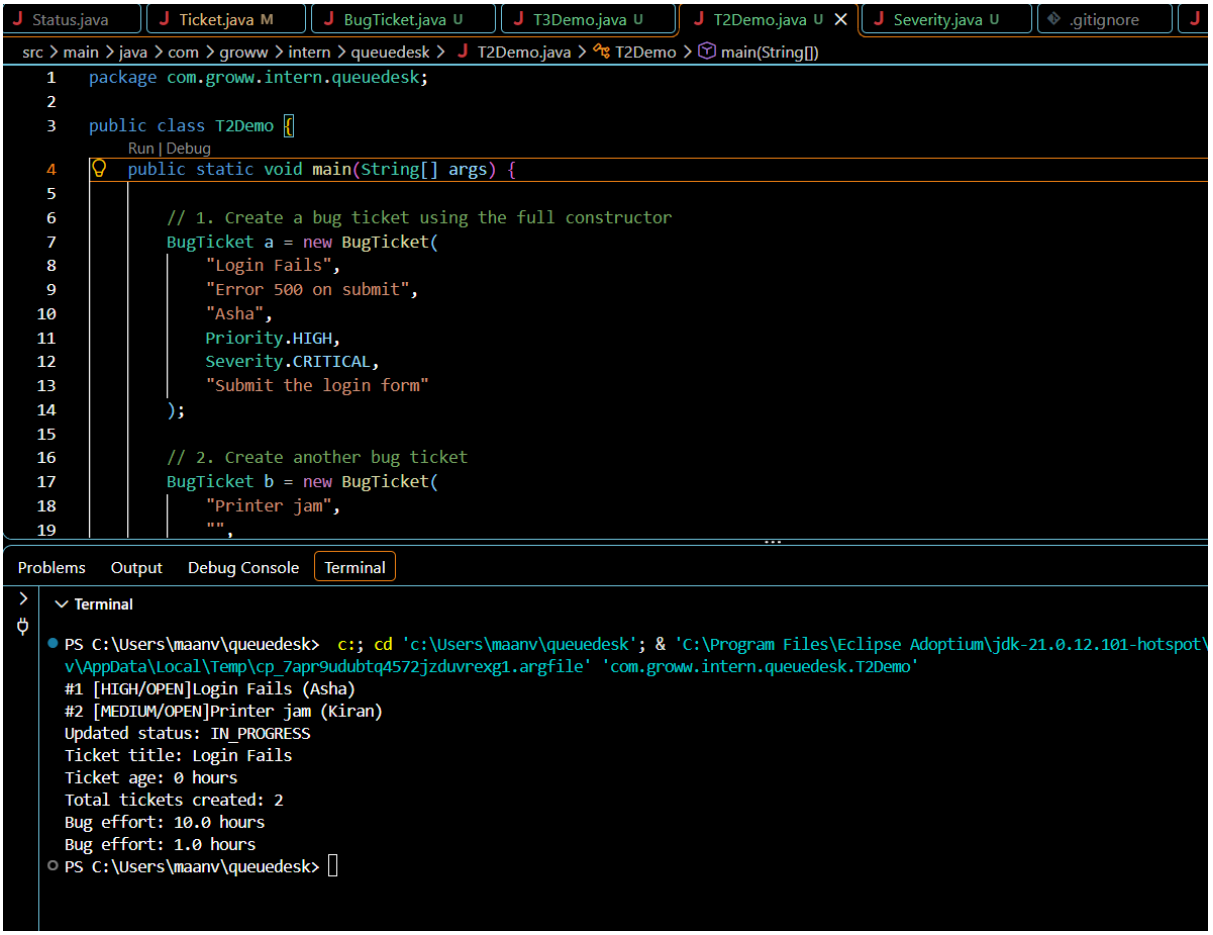
        // 5. Test the title getter
        System.out.println("Ticket title: " + a.getTitle());

        // 6. Test ticket age
        System.out.println("Ticket age: " + a.ageInHours() + " hours");

        // 7. Test total tickets created
        System.out.println("Total tickets created: " +
Ticket.totalCreated());

        // 8. Test effort estimation
        System.out.println("Bug effort: " + a.estimateEffortHours() + "
hours");
        System.out.println("Bug effort: " + b.estimateEffortHours() + "
hours");
    }
}
```

T2Demo.java


Status.java
Ticket.java M
BugTicket.java U
T3Demo.java U
T2Demo.java U
Severity.java U
.gitignore
src > main > java > com > groww > intern > queuedesk > T2Demo.java > T2Demo > main(String[])
1 package com.groww.intern.queuedesk;
2
3 public class T2Demo {
Run | Debug
4 public static void main(String[] args) {
5
6 // 1. Create a bug ticket using the full constructor
7 BugTicket a = new BugTicket(
8 "Login Fails",
9 "Error 500 on submit",
10 "Asha",
11 Priority.HIGH,
12 Severity.CRITICAL,
13 "Submit the login form"
14);
15
16 // 2. Create another bug ticket
17 BugTicket b = new BugTicket(
18 "Printer jam",
19 "",
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\
v\AppData\Local\Temp\cp_7apr9udubtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T2Demo'
#1 [HIGH/OPEN]Login Fails (Asha)
#2 [MEDIUM/OPEN]Printer jam (Kiran)
Updated status: IN_PROGRESS
Ticket title: Login Fails
Ticket age: 0 hours
Total tickets created: 2
Bug effort: 10.0 hours
Bug effort: 1.0 hours
PS C:\Users\maanv\queuedesk>

BugTicket.java

```java
package com.groww.intern.queuedesk;

import java.util.Objects;

//Build-T3 Req3: BugTicket Subclass; extends Ticket means BugTicket
inherits all the fields and methods of Ticket, Also added it's own
severity and stepsToReproduce fields. The constructor initializes all
the fields, including those inherited from Ticket. The
estimateEffortHours() method is implemented to provide a specific
effort estimation for bug tickets based on their severity.
public class BugTicket extends Ticket{
    //bug specific fields
    private final Severity severity;
    private final String stepsToReproduce;

    // Constructor: initializes both common Ticket data and
bug-specific data.
    public BugTicket(String title, String description, String
requester, Priority priority, Severity severity, String
stepsToReproduce){

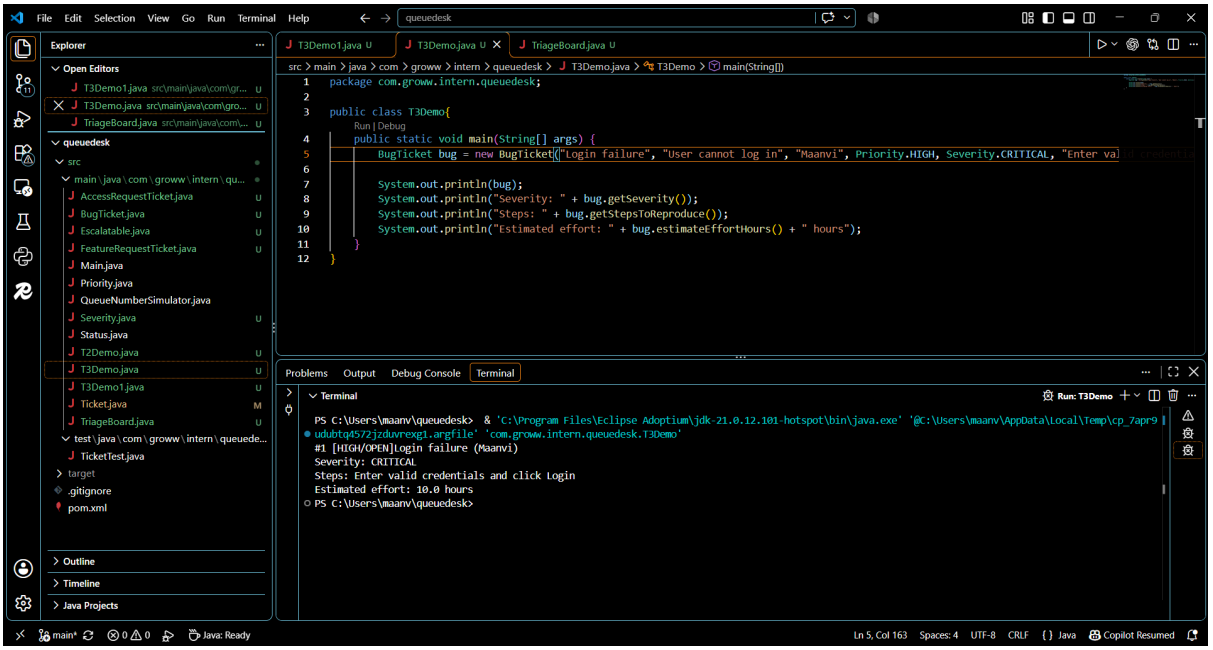
        //calls the parent class constructor, must be first statement
in this constructor.
        super(title, description, requester, priority);

        this.severity = Objects.requireNonNull(severity, "Severity");
        // If steps are null, store an empty string; otherwise, remove
extra spaces.
        this.stepsToReproduce = stepsToReproduce == null? "" :
stepsToReproduce.trim();
    }
    //our fields are private, so we need to use getters and setters to
access them.
    public Severity getSeverity(){
        return severity;
    }
    public String getStepsToReproduce(){
        return stepsToReproduce;
```

```
    }

    //implement the abstract method from Ticket to estimate effort
based on severity.
    @Override
    public double estimateEffortHours(){
        switch(severity){
            case LOW:
                return 1.0;
            case MEDIUM:
                return 3.0;
            case HIGH:
                return 6.0;
            case CRITICAL:
                return 10.0;
            default:
                throw new IllegalStateException("Unknown severity");
        }
    }
}
```

T3Demo.java



- FeatureRequestTicket extends Ticket — adds votes and businessJustification.
- AccessRequestTicket extends Ticket — adds systemName and accessLevel.
- Interface Escalatable: void escalate(); boolean isEscalated(); int escalationLevel(); plus one default method, e.g. default String escalationBadge(). Implement it on BugTicket and AccessRequestTicket only.

FeatureRequestTicket.java

```java
//Build:T3 - Req3: A ticket for requesting a new feature.

package com.groww.intern.queuedesk;

import java.util.Objects;

// votes = how many users want this feature.
// businessJustification = why implementing it is valuable.
public class FeatureRequestTicket extends Ticket {

    // Feature-specific fields
    private int votes;
    private final String businessJustification;

    // Constructor: initializes common and feature-specific fields.
```

```
    public FeatureRequestTicket(String title, String description,
String requester, Priority priority, int votes, String
businessJustification) {

        // Initialize the common Ticket fields.
        super(title, description, requester, priority);

        // Votes cannot be negative.
        if (votes < 0) {
            throw new IllegalArgumentException("Votes cannot be
negative");
        }
        this.votes = votes;

        // Business justification cannot be null.
        this.businessJustification =
Objects.requireNonNull(businessJustification, "Business justification
cannot be null").trim();
    }

    public int getVotes() {
        return votes;
    }

    // Updates votes, ensuring the value is not negative.
    public void setVotes(int votes) {
        if (votes < 0) {
            throw new IllegalArgumentException("Votes cannot be
negative");
        }
        this.votes = votes;
    }

    public String getBusinessJustification() {
        return businessJustification;
    }

    // Provides an effort estimate for a feature request.
    @Override
    public double estimateEffortHours() {
```

```
        return 8.0;
    }
}
```

AccessRequestTicket.java

```
package com.groww.intern.queuedesk;

import java.util.Objects;

// T3 Req4: A ticket for requesting access to a system.
public class AccessRequestTicket extends Ticket implements Escalatable{

    // Fields specific to access requests
    private final String systemName;
    private final String accessLevel;

    // Tracks whether the ticket has been escalated.
    private boolean escalated = false;

    // Tracks the current escalation level.
    private int escalationLevel = 0;

    // Constructor initializes common and access-specific fields.
    public AccessRequestTicket(String title, String description, String
requester, Priority priority, String systemName, String accessLevel) {

        // Initialize common Ticket fields.
        super(title, description, requester, priority);

        // System name and access level cannot be null.
        this.systemName = Objects.requireNonNull(systemName, "System
name cannot be null").trim();

        this.accessLevel = Objects.requireNonNull(accessLevel, "Access
level cannot be null").trim();
    }

    public String getSystemName() {
        return systemName;
```

```
    }

    public String getAccessLevel() {
        return accessLevel;
    }

    //Provides an effort estimate for an access request.
    @Override
    public double estimateEffortHours() {
        return 2.0;
    }
    // Implement the Escalatable interface methods.
    @Override
    public void escalate(){
        escalated = true;
        escalationLevel++;
    }

    @Override
    public boolean isEscalated(){
        return escalated;
    }

   @Override
    public int escalationLevel(){
        return escalationLevel;
    }
}
```

Escalatable.java

```
//An interface defines what a class must be able to do, and not how it
does it.In this case, the Escalatable interface defines the behavior of
tickets that can be escalated. Any class that implements this interface
must provide implementations for the methods defined in the interface.
package com.groww.intern.queuedesk;

// T3 Req5: Defines the behavior of tickets that can be escalated.
public interface Escalatable {
```

```
    // Escalate the ticket to a higher level.
    void escalate();

    // Check whether the ticket has been escalated.
    boolean isEscalated();

    // Returns the current escalation level.
    int escalationLevel();

    // Default method shared by classes implementing this interface.
    default String escalationBadge() {
        if (isEscalated()) {
            return "[ESCALATED - Level " + escalationLevel() + "]";
        }
        return "[NOT ESCALATED]";
    }
}
```

- Write printTriageBoard(List<Ticket> tickets) that loops over the base type calling estimateEffortHours() and toString(), then uses if (ticket instanceof Escalatable e) to conditionally show escalation state.

TriageBoard.java

```
package com.groww.intern.queuedesk;

import java.util.List;
/*Imagine QueueDesk has three tickets:
Ticket 1 — BugTicket: Login failure, effort = 10 hours, supports
escalation.
Ticket 2 — FeatureRequestTicket: Add expense report, effort = 8 hours,
does not support escalation.
Ticket 3 — AccessRequestTicket: Database access, effort = 2 hours,
supports escalation.

We want one method that prints all three tickets without writing
separate printing code for each type. That is why printTriageBoard()
exists. */
//T3 Req6: Displays different ticket types on one triage board.
public class TriageBoard {
```

```java
    //Create a method called printTriageBoard that takes a list of
tickets and prints them all.Why Ticket? becoz all three classes extend
ticket.Accepts a list containing different types of Ticket subclasses.
    //List<Ticket> accepts a list of tickets. All 3 classes extend
Ticket, so their objects can be stored in the same list.
    public static void printTriageBoard(List<Ticket> tickets) {

        //loop through tickets- for every ticket in the list, process
one ticket at a time.
        for (Ticket ticket : tickets) {

            //Print the current ticket's details using its toString()
method.
            System.out.println(ticket.toString());

            //Calls the effort-estimation method implemented by the
actual ticket subclass. This is polymorphism: each ticket type can
return a different value.
            //For example, a BugTicket with CRITICAL severity returns
10.0 hours, while an AccessRequestTicket returns 2.0 hours in our
current implementation.
            System.out.println("Estimated effort: " +
ticket.estimateEffortHours() + " hours");

            //Check whether this ticket supports escalation.
            //if (ticket instanceof Escalatable e) — checks whether the
ticket implements Escalatable. If yes, e refers to that ticket through
the interface.
            if (ticket instanceof Escalatable e) {

                //e.escalationBadge() — gets the ticket's escalation
status and prints it.
                System.out.println("Escalation: " +
e.escalationBadge());
            }

            //prints a separator between tickets to make the output
easier to read.
            System.out.println("-------------------------");
```

```
        }
    }
}
```

T3Demo1.java

```
package com.groww.intern.queuedesk;

import java.util.ArrayList;
import java.util.List;

//Build T3 Demo: Test the triage board with different ticket types.
public class T3Demo1{
    public static void main(String[] args){

        //Create a BugTicket, parameters: title, description,
requester, priority, severity, stepsToReproduce
        BugTicket bug = new BugTicket("Login failure", "User cannot log
in", "Maanvi", Priority.HIGH, Severity.CRITICAL, "Enter valid
credentials and click Login");

        //Create a FeatureRequestTicket, parameters: title,
description, requester, priority, votes, businessjustification
        FeatureRequestTicket feature = new FeatureRequestTicket("Add
expense reports", "Allow users to download monthly reports", "Asha",
Priority.MEDIUM, 25, "Helps users track their spending");

        //Create an AccessRequestTicket, parameters: title,
description, requester, priority, systemName, accessLevel
        AccessRequestTicket access = new AccessRequestTicket("Database
access", "Request read-only access", "Rahul", Priority.HIGH,
"Production Database", "Read-only");

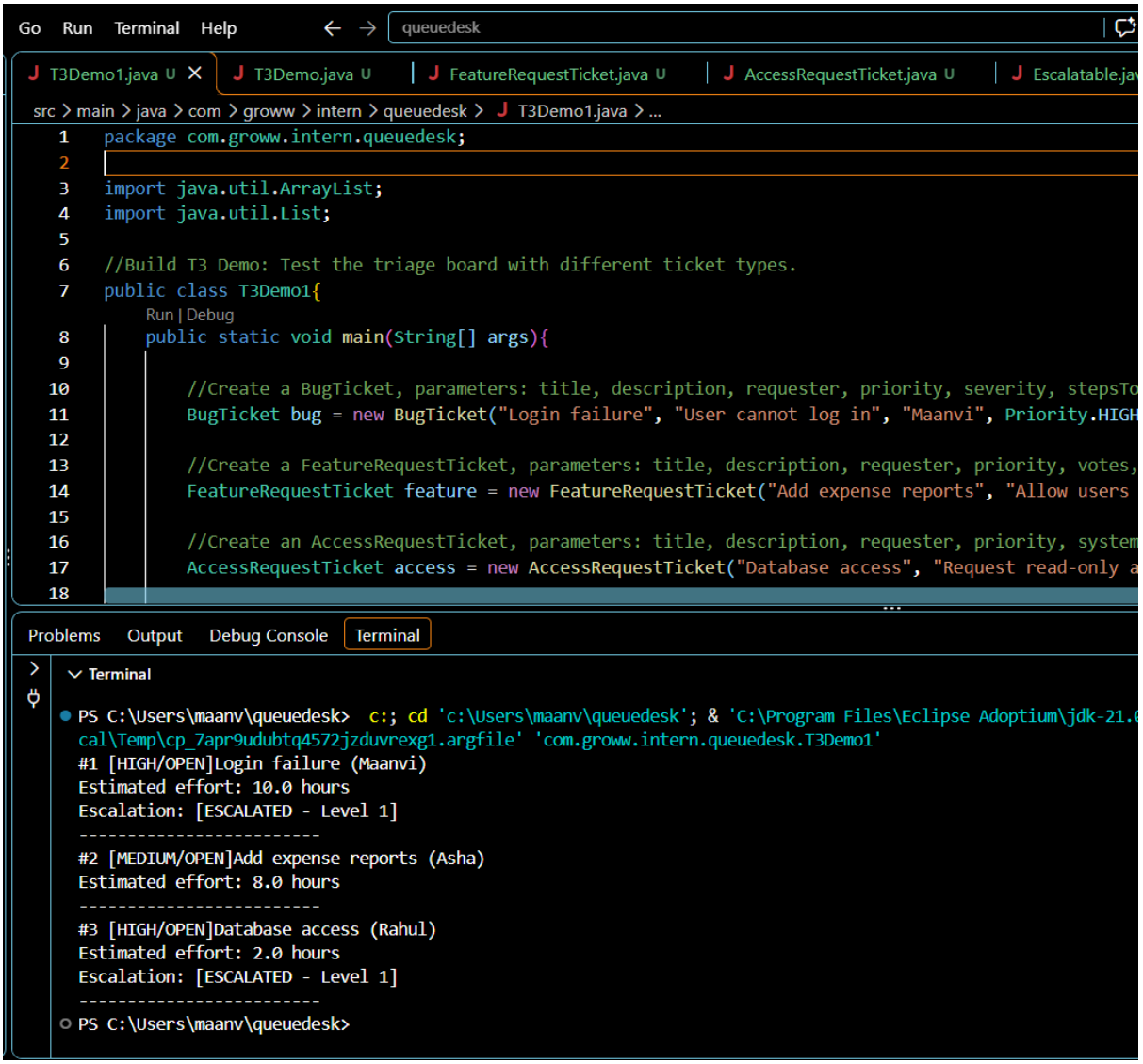
        //Escalate the bug ticket and access request.
        bug.escalate();
        access.escalate();

        //Create one list that can hold all three types of tickets.
Since all three classes extend Ticket, we can store them in a
List<Ticket>.
```

```
        List<Ticket> tickets = new ArrayList<>();
        //Add the tickets to the list.
        tickets.add(bug);
        tickets.add(feature);
        tickets.add(access);

        //Print the triage board with all tickets.
        TriageBoard.printTriageBoard(tickets);
    }
}
/*
// T4Demo.java — CREATE the list
List<Ticket> tickets = new ArrayList<>();
tickets.add(bug);
tickets.add(feature);
tickets.add(access);

// TriageBoard.java — RECEIVE and PROCESS the list
public static void printTriageBoard(List<Ticket> tickets) {
    for (Ticket ticket : tickets) {
        System.out.println(ticket);
    }
} */
```



Java Collections Framework, Generics & Sorting

Month 1 — Build T4: Repository and Triage Queue

1. Java Collections Framework (JCF)

Definition

- A framework in `java.util` that provides interfaces and implementations to store, retrieve, manipulate and process groups of objects.
- Provides reusable data structures instead of requiring developers to implement them from scratch.
- Uses interfaces such as `List`, `Set`, `Queue` and `Map`.
- Most collections are generic, providing compile-time type safety.

BUILD-T4

- Build: repository and triage queue
  - Add assignee (nullable String) and tags (Set<String>) fields to Ticket, with a small setter/adder for each.
  - Make Ticket implements Comparable<Ticket>: order by priority descending, then by createdAt ascending for ties.

Ticket.java

```
import java.util.Set; //Set stores unique elements.
import java.util.HashSet; //HashSet is a collection that implements the Set interface.So, one implementation of Set.
import java.util.Collections; //provides a read-only view of a collection. Collections.unmodifiableSet() returns an unmodifiable view of the specified set. Any attempt to modify the set through this view will result in an UnsupportedOperationException.
```

Then

```
public abstract class Ticket implements Comparable<Ticket>{ //Build-T4 Req 2: Comparable<Ticket> means the ticket class defines how two tickets should be compared for natural ordering.
```

Added,

```
private String assignee; //Build-T4 Req 1: Add a field to store the assignee of the ticket. This field is optional and can be null if the ticket is not assigned to anyone yet.
    private final Set<String> tags = new HashSet<>(); //Build-T4 Req 1: Add a field to store the tags(are labels such as "urgent", "backend") associated with the ticket. This field is a set of strings, which means it can store multiple unique tags. We use HashSet to ensure that each tag is unique and to provide efficient operations for adding, removing, and checking for tags.
```

```java
 //Build-T4 Req1: Add methods

    //Getter: returns the assignee of the ticket. If the ticket is not
assigned to anyone, it returns null.
    public String getAssignee(){
        return assignee;
    }

    //Setter: assign the ticket to a specific person.
    public void setAssignee(String assignee){
        this.assignee = assignee;
    }

    //Getter: returns a read-only view of the tags.This prevents
external code from modifying the internal set of tags directly,
ensuring encapsulation.
    public Set<String> getTags(){
        return Collections.unmodifiableSet(tags);
    }

    /*Setter: add a tag to the ticket.
    first replaces the existing tags with the provided set, we validate
each tag through addTag() instead of directly assigning the incoming
set, protecting the object's data..
    */
    public void setTags(Set<String> newTags){
        this.tags.clear(); //remove the existing tags before adding the
new ones.

        if(newTags == null){
            return; //if null is provided, leave the set empty.
        }

        //add each tag through addTag(), which validates the tag before
adding it to the set.
        for(String tag: newTags){ //takes each tag from the newTags set
and passes it to the addTag() method for validation and addition to the
internal tags set.
            this.addTag(tag); //addTag() method checks if the tag is
valid (not null or blank) before adding it to the internal tags set.
```

```
            // Checks tag is not null, empty, or just spaces.Then
removes extra spaces using trim() and adds it to the set. If valid,
adds it to the tags set. If invalid, ignores it.
        }
        /*If the incoming set contains " urgent ", addTag() stores it
as "urgent". If it contains "   ", the method throws an exception. */
    }
        //Adder: adds one tag at a time. Set prevents duplicate tags,
while validation prevents null, empty or whitespaces-only tags.
    public void addTag(String tag){
        if(tag == null || tag.isBlank()){ // If a tag is invalid,
addTag() throws an IllegalArgumentException.
            throw new IllegalArgumentException("Tag cannot be null or
blank");
        }
        //removes leading and trailing spaces from the tag before
adding it to the set.
        tags.add(tag.trim());
    }

    /*Build-T4 Req2: Defines the natural/default sorting order of
tickets.
    ■ Make Ticket implements Comparable<Ticket>: order by priority
descending, then by
    createdAt ascending for ties.
    Means 1.Higher priority tickets comes first: CRITICAL → HIGH →
MEDIUM → LOW.
    2.TIE, If priorities are equal, the older ticket (createdAt) comes
first.*/
    @Override
    public int compareTo(Ticket other){ //the Comparable<Ticket>
interface requires us to implement this method.
    /*Here,
    other= the second ticket being compared to the current ticket
(this).
    this = the current ticket.
    this.priority = priority of the current ticket (this).
    other.priority = priority of the other ticket being compared
(other).
    this.createdAt = creation time of the current ticket (this).
```

```
    other.createdAt = creation time of the other ticket being compared
(other).
    ordinal() = the numerical position of that priority in the enum.
    enum Priority {
    LOW, MEDIUM, HIGH, CRITICAL
    }

        //Compare priorities in descending order (CRITICAL → HIGH →
MEDIUM → LOW).
        Priority    ordinal()
           LOW       0
           MEDIUM    1
           HIGH      2
           CRITICAL  3
        Integer.compare(a, b): Is a shortcut for comparing two integers
instead of writing if-else conditions manually. if(a>b) return 1;
if(a<b) return -1; else return 0;{so negative if the first number is
smaller, zero if both numbers are equal and positive if the first
number is greater.}
        return Integer.compare(a,b); both expresses the same comparison
logic
        How do we sort? Aesc order: Integer.compare(a,b)
                        Desc order: Integer.compare(b,a)
        Now imagine:
        - Current ticket (this) has LOW → 0
        - Other ticket (other) has CRITICAL → 3
        becomes: Integer.compare(3, 0); The result is positive because
3 > 0.
        Why put other first? Because we want CRITICAL to come before
LOW.
        Reversing the arguments puts higher priorities first.*/
        int priorityComparison =
Integer.compare(other.priority.ordinal(), this.priority.ordinal());

        //if priorities are different, return the comparison result.
        if(priorityComparison != 0){
            return priorityComparison;
        }
```

```
        //if priorities are equal, compare creation times i.e.
createdAt in ascending order (older tickets first).
        return this.createdAt.compareTo(other.createdAt);
    }
```

- Write a TicketComparators utility with at least two Comparator<Ticket> built via Comparator.comparing(...).thenComparing(...) — e.g. by assignee then priority, and by status then age.

Comparator — Custom Ordering

Definition

- `Comparator` is an interface in `java.util`.
- Defines an external or alternative comparison rule.
- Allows the same class to have multiple sorting strategies.
- Uses `compare(a, b)`.

Comparable vs Comparator

Comparable	Comparator
Natural ordering	Custom ordering
`compareTo()`	`compare()`
Implemented by the class	Separate comparison strategy
Usually one default ordering	Multiple alternative orderings
`java.lang`	`java.util`

Comparator utility in QueueDesk

Created `TicketComparators` to define alternative sorting rules without changing the Ticket class.

TicketComparator.java

```
package com.groww.intern.queuedesk;
/*One sorting rule cannot conveniently handle every use case. That's
why we have created TicketComparators.java.
ticket1.compareTo(ticket2) --> Defines the default order: priority
descending, then creation time ascending.
*/
import java.util.Comparator;
//Build-T4 Req3: Provides alternative ways to sort tickets.
public class TicketComparators{
    //Comparator 1: Sort by assignee alphabetically, then by priority
from highest to lowest.
    /*Comparator<Ticket> -  A rule for comparing tickets
    BY_ASSIGNEE_THEN_PRIORITY - The name of the rule
    Ticket::getAssignee - Get the ticket's assignee
    Comparator.nullsFirst(...) - put tickets with no assignee before
tickets with an assignee.
    Comparator.naturalOrder() - Sort names alphabetically
    .thenComparing(...) - If the first comparison is tied, use another
rule
    Ticket::getPriority - Get the ticket's priority
    Priority::ordinal - Get the priority's position in the enum
    .reversed() - Reverse the order so higher enum positions come first
    Comparator.comparingInt(Priority::ordinal).reversed()) - Compare
priorities by their enum position, with higher positions coming first.
*/


    public static final Comparator<Ticket> BY_ASSIGNEE_THEN_PRIORITY =
Comparator.comparing(Ticket::getAssignee,
Comparator.nullsFirst(Comparator.naturalOrder())).thenComparing(Ticket:
:getPriority, Comparator.comparingInt(Priority::ordinal).reversed());
```

```
    //Comparator 2 : Sort by status first, then by ticket age(oldest
creation time first).
    public static final Comparator<Ticket> BY_STATUS_THEN_AGE =
Comparator.comparing(Ticket::getStatus).thenComparing(Ticket::getCreate
dAt);
}
```

- Write a generic class Repository<T extends Ticket> wrapping a private List<T>: add(T), all() (returning an unmodifiable view), findById(int), and groupByAssignee() returning Map<String, List<T>>.

Generics

```
## 1. WHAT ARE GENERICS IN JAVA?
1. In java, Generics are Parameterized types introduced in Java5 to
allow classes, interfaces and methods to accept data types as
parameters.
So, it allows us to write classes, interfaces and methods that can work
with different data types.
They provide a way to reuse code with different inputs while enforcing
strict type-checking at compile time.

They use type parameters (placeholders) such as T, E, K and V.
• T - Type (General purpose)
• E - Element (Heavily used by the Java Collections Framework, like
List<E>)
• K - Key (Used in maps, like Map<K, V>)
• V - Value (Used in maps)

The actual data type is specified when the class or method is used.

```java
List<String> names = new ArrayList<>();
```
* **List**: The container (a collection).
* **<String>**: The label. It tells the compiler, *"This list can ONLY
hold pieces of text (Strings)."*
* **Result**: You can add `"Hello"`, but if you try to add the number
`100`, the system will stop you immediately.
```

---

2. What was the scenario before Generics?
Before Java 5, containers didn't have specific labels. They were treated as generic "Object" boxes that could hold absolutely anything mixed together.

```java
// Before Generics
List list = new ArrayList();

list.add("Hello");  // A String (Text)
list.add(100);      // An Integer (Number)
list.add(20.5);     // A Double (Decimal)
```

Why this caused massive problems:

Problem 1: No Type Safety
Because the list accepted everything, the computer couldn't double-check your work.
A collection could accept different types of objects. If you accidentally put a number into a list meant for names, nobody would notice until the application crashed while someone was using it.

Problem 2: Manual typecasting was required
When pulling something out of the box, Java didn't remember what it was. You had to manually tell it what data type to expect.
This was called explicit typecasting.
If we cast an object to an incompatible type, a ClassCastException could occur at runtime.

```java
String name = (String) list.get(0);  // Works fine because "Hello" is text.
String value = (String) list.get(1); // CRASH! Throws a 'ClassCastException'.
```

```
* **Why the crash?** `list.get(1)` is the number `100`. You cannot
force a computer to convert a mathematical number directly into a text
string like that without extra steps.
Why does the second line fail?
- list.get(1) returns an Integer object.
- We are trying to convert it into a String.
- This causes a ClassCastException.

## Problem 3: More conditional checks and code complexity
To avoid crashes, developers had to write repetitive logic to
constantly check the "identity" of every item before touching it:

Developers sometimes used instanceof to check the type of each object.
They then manually cast the object to the appropriate type.
More data types meant more conditions and additional code.
This made programs harder to maintain.

```java
for (Object item : list) {
    if (item instanceof String) {
        // Safe to process as text
    } else if (item instanceof Integer) {
        // Safe to process as a number
    }
}
```
Problem 4: Code duplication
Similar classes and methods had to be written separately for different
data types.
This reduced code reusability and increased maintenance effort.

3. How did Generics solve these problems?
Generics introduced the concept of specifying the data type while
declaring a collection or creating an object.

```java
// After Generics
List<String> names = new ArrayList<>();

names.add("Hello");
```
```

```
names.add("Java");
// names.add(100); <-- The code won't even compile! It blocks you
instantly.
```
Only String objects can be added.
The compiler checks the data type at compile time.
Incompatible data types are rejected before program execution.
We do not need manual typecasting when retrieving elements.

```java
String name = names.get(0); //we don't need (String) anymore.
```

---

4. Before Generics vs. After Generics (Comparison Matrix)

Feature / Scenario	Before Generics (Old Java)	After Generics (Modern Java)
Storage Flexibility	Collections could store mixed, random object types together.	Specify the required type for each collection.
Data Retrieval	Manual typecasting required when retrieving elements.	No manual typecasting needed; Java retrieves the specified type.
Error Timing	Type-related errors could occur at runtime.	Type mismatches are caught at compile time.
Code Inspection	More `instanceof` checks required to identify object types.	Fewer type checks because the collection has a defined type.
Code Maintenance	More code duplication for different data types.	Better code reusability; write logic once and reuse it with different types.
Type Safety	Less type safety, increasing the risk of type-related bugs.	Improved type safety through compile-time checking.

5. Why are Generics Important?

1. Type Safety
   - Restricts the type of data that can be stored.
   - Detects incompatible types at compile time.

2. Code Reusability
   - Write one generic class or method and reuse it with different data types.
   - Avoids writing separate implementations for every type.
3. No Manual Typecasting
   - Retrieved values are already of the specified type.
   - Reduces casting-related errors.
4. Compile-Time Error Detection
   - Type mismatches are identified by the compiler rather than waiting for runtime.
5. Better Readability and Maintainability
   - Makes code easier to understand.
   - Clearly communicates which types a class or method accepts.
6. Reduced Code Duplication
   - A single generic implementation can handle multiple permitted types.

-- Before Generics, Java collections could hold objects without specifying their types, so developers often had to manually typecast retrieved values and handle type mismatches. Generics introduced parameterized types, allowing developers to specify the expected data type, improve type safety, reduce casting, and write reusable code.

What is T in Generics?
- T is a type parameter.
- It acts as a placeholder for an actual data type.
- The actual type is provided when the generic class is instantiated.
- T is a convention; technically, another valid identifier can be used.

Example:

```
class Box<T> {

    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
```

```
}


Using the class:
Box<String> box1 = new Box<>();
Box<Integer> box2 = new Box<>();


- In Box<String>, T becomes String.
- In Box<Integer>, T becomes Integer.
- The same class works with different data types.

## Bounded Type Parameters
Sometimes we don't want a generic class to accept every possible data
type.

We can restrict the permitted types using the extends keyword.

Syntax:
class ClassName<T extends SomeClass> {
}


Meaning:
- T must be the specified class or one of its subclasses.
- This is called an upper-bounded type parameter.
Example from QueueDesk:
public class Repository<T extends Ticket> {
    private List<T> tickets = new ArrayList<>();
}


Here:
- T is the placeholder.
- extends Ticket restricts T to Ticket or its subclasses.
- The repository can store BugTicket, FeatureRequestTicket, etc.
- It cannot store unrelated objects such as String or Integer.

## Generic Methods
A generic method is a method that declares its own type parameter.
```

```
Syntax:
public static <T> void display(T value) {
    System.out.println(value);
}


- <T> declares the type parameter.
- T is the return/input type.
- The method can work with different data types.
Example:
display("Hello");
display(100);
display(20.5);


QueueDesk example:
public static <T extends Ticket> T oldest(List<T> tickets)


- Accepts a list of Ticket or its subclasses.
- Returns an object of the same generic type T.
- Allows the method to work with different ticket types.

## What is the difference between T and ?
- T is a named type parameter.
- ? is a wildcard representing an unknown type.
- T is used when we want to refer to the same type at multiple places.
- ? is useful when we only need to work with a collection of an unknown type.
Example:
List<T> tickets;
List<?> items;

Wildcards can also have bounds, such as ? extends Ticket and ? super Ticket. Learn these after understanding basic Generics.

## What is Type Erasure?
- Generics are primarily a compile-time feature in Java.
```

```
- During compilation, the compiler performs type checks and then
removes or erases most generic type information.
- This process is called Type Erasure.
- It helps maintain compatibility with older Java code.
For example:
List<String> names = new ArrayList<>();

The runtime representation is essentially a regular List; the generic
type information is not generally retained as a runtime type parameter.

12. QueueDesk – Where am I using Generics?
Code    Purpose
List<Ticket>    Stores Ticket objects
Set<String> Stores String tags
Map<String, List<T>>    Maps an assignee to a list of tickets
Repository<T extends Ticket>    Generic repository restricted to Ticket
types
<T extends Ticket> T oldest(...)    Generic method returning the oldest
ticket
```

Repository.java

```
package com.groww.intern.queuedesk;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

//Build-T4 Req4: Repository class to store tickets and provide methods.
public class Repository<T extends Ticket> {//bounded type parameter ,
it restricts T so that this repository can only be used for the Ticket
class or any subclass of Ticket class.
    private final List<T> tickets = new ArrayList<>();
   /*private as we don't want other classes to directly access or
modify our internal ticket list.
   Instead, they should use the methods we provide:
   - add() to add tickets.
   - all() to view tickets.
   - findById() to search.
```

```
   - groupByAssignee() to group.
    This is an example of Encapsulation.
    List is an interface in java, it represents an ordered collection
of elements , <T> means the list stores objects of the type represented
by T.
    new ArrayList<>() --> creates an empty list in memory.

    private final List<T> tickets = new ArrayList<>(); creates a
private, initially empty, dynamically resizable list to store tickets
of type T; private provides encapsulation, final prevents reassignment
of the list reference, and List<T> ensures type safety.*/

    //Req1: add(T ticket) method to add a ticket to the repository.
    public void add(T ticket){ //accepts a ticket and adds it to our
internal list i.e. tickets.
        tickets.add(ticket);
    }

    //Req2: all() method to return all tickets in the repository.
    public List<T> all(){
        return Collections.unmodifiableList(tickets); //returns an
unmodifiable view of the tickets list, preventing external
modification.
    }

    //Req3: findById(int id) method to find a ticket by its ID.
    public T findById(int id){
        for(T ticket : tickets){ //Iterates through each ticket in the
tickets list.
            if(ticket.getId() == id){ //Checks if the ticket's ID
matches the provided ID.
                return ticket; //If a match is found, returns the
ticket.
            }
        }
        return null; //If no ticket with the given ID is found, returns
null.
    }
```

```
    //Req4: groupByAssignee() method to group tickets by their
assignee.
    //key = assignee(name-string), value = list of tickets assigned to
that assignee.

    public Map<String, List<T>> groupByAssignee(){
        Map<String, List<T>> groupedTickets = new HashMap<>();
//Creates an empty HashMap to store the grouped tickets.
        for(T ticket : tickets){ //Iterates through each ticket in the
tickets list.
            String assignee = ticket.getAssignee(); //Retrieves the
assignee of the current ticket.
            // groupedTickets.computeIfAbsent(assignee, k -> new
ArrayList<>()).add(ticket); //If the assignee is not already a key in
the map, creates a new list for that assignee and adds the ticket to
it.

            //If the assignee is not already a key in the map, create a
new list for that assignee and add the ticket to it.
            if(!groupedTickets.containsKey(assignee)){ //Checks if the
assignee is already a key in the map.
                groupedTickets.put(assignee, new ArrayList<>()); //If
not, creates a new list for that assignee.
            }
            groupedTickets.get(assignee).add(ticket); //Adds the
current ticket to the list of tickets for that assignee.
        }
        return groupedTickets; //Returns the map of grouped tickets.
    }

}
```

- Build a PriorityQueue<Ticket> triage queue seeded from natural ordering, plus a second one constructed with a Comparator from TicketComparators — offer()/poll() to serve the next ticket either way.

```
# Priority Queue in Java

## 1. What is a Priority Queue?
```

- A Priority Queue is a data structure in which elements are served according to their priority rather than insertion order.
- The element with the highest priority according to the defined ordering is served first.
- It is generally implemented using a **Binary Heap**.
- Insertion and deletion take O(log n) time.
- Accessing the head element takes O(1) time.

---

2. Priority Queue in C++ vs Java

C++ Syntax

```cpp
#include <queue>
using namespace std;

priority_queue<int> pq;

// Insert
pq.push(10);

// Remove highest-priority element
pq.pop();

// Access top element
pq.top();

// Check empty
pq.empty();
```

Important:
- C++ Priority Queue is a **Max Heap by default**.
- Largest element comes first.

Java Syntax

```java
import java.util.PriorityQueue;

PriorityQueue<Integer> pq = new PriorityQueue<>();

// Insert
pq.offer(10);

// Remove head element
pq.poll();

// Access head element
pq.peek();

// Check empty
pq.isEmpty();
```

Important:
- Java PriorityQueue is a **Min Heap by default**.
- Smallest element comes first.

---

3. C++ vs Java – Important Syntax Changes

Remember these conversions:

1. C++ `push()` → Java `offer()`
2. C++ `pop()` → Java `poll()`
3. C++ `top()` → Java `peek()`
4. C++ `empty()` → Java `isEmpty()`
5. C++ `size()` → Java `size()`

Major difference:

- C++ gives Max Heap by default.
- Java gives Min Heap by default.
- Java allows us to define custom ordering using Comparator.

---

4. Types of Priority Queues in Java

A. Min Heap (Default)

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.offer(30);
pq.offer(10);
pq.offer(20);

System.out.println(pq.poll()); // 10
```

Explanation:
- Java uses natural ascending order by default.
- The smallest element is removed first.

B. Max Heap

```java
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Comparator.reverseOrder());

pq.offer(30);
pq.offer(10);
pq.offer(20);

System.out.println(pq.poll()); // 30
```

Explanation:
- `Comparator.reverseOrder()` reverses the natural ordering.
- The largest element is removed first.

---

5. Important PriorityQueue Methods

1. offer(element)
- Inserts an element into the queue.
- Similar to `push()` in C++.

2. poll()
- Removes and returns the head element.
- Returns null if the queue is empty.
- Similar to `pop()` in C++, but unlike C++, it also returns the removed element.

3. peek()
- Returns the head element without removing it.
- Similar to `top()` in C++.

4. isEmpty()
- Checks whether the queue is empty.

5. size()
- Returns the number of elements.

6. clear()
- Removes all elements from the queue.

---

6. Custom Priority Queue Using Comparator

Sometimes we don't want the default ordering.

We can define our own ordering using a Comparator.

Example: Max Heap using Comparator.

```java
PriorityQueue<Integer> pq =
    new PriorityQueue<>((a, b) -> Integer.compare(b, a));
```

Explanation:

- The comparator decides which element should come first.
- `Integer.compare(b, a)` reverses the natural ascending order.
- Therefore, the largest element comes first.

---

7. Comparable vs Comparator in PriorityQueue

Comparable

- Defines the natural ordering of objects.
- Implemented inside the class itself.
- Uses the `compareTo()` method.

Example:

```java
public class Ticket implements Comparable<Ticket> {
    @Override
    public int compareTo(Ticket other) {
        // Define natural ordering here.
    }
}
```

Comparator

- Defines custom ordering externally.
- Does not require changing the original class.
- Uses the `compare()` method or comparator-building methods.

Example:

```java
Comparator<Ticket> byPriority =
    Comparator.comparing(Ticket::getPriority);
```

Remember:
- Comparable → Natural ordering.

- Comparator → Custom ordering.

---

8. PriorityQueue in QueueDesk Project

We are maintaining TWO priority queues.

Queue 1: Natural Ordering

```java
private final PriorityQueue<Ticket> naturalQueue =
    new PriorityQueue<>();
```

How does it work?

- Uses the `compareTo()` method implemented in `Ticket.java`.
- Higher priority tickets are served first.
- If priorities are equal, the older ticket is served first.

Ordering sequence:

1. CRITICAL
2. HIGH
3. MEDIUM
4. LOW

For tickets with equal priority, `createdAt` decides the order, with the older ticket first.

Queue 2: Custom Ordering

```java
private final PriorityQueue<Ticket> customQueue =
    new PriorityQueue<>(TicketComparators.BY_ASSIGNEE_THEN_PRIORITY);
```

How does it work?

- Uses the Comparator defined in `TicketComparators.java`.
- First sorts by assignee in alphabetical order.
- If assignees are equal, higher priority comes first.
- Null assignees are handled using `Comparator.nullsFirst()`.

Important: This queue does not necessarily serve the highest-priority ticket globally first because assignee ordering comes first.

---

9. Adding and Removing Tickets

offer() – Add ticket

```java
naturalQueue.offer(ticket);
customQueue.offer(ticket);
```

- Adds the ticket to both queues.
- Each queue independently maintains its own ordering.

poll() – Serve next ticket

```java
Ticket next = naturalQueue.poll();
```

- Removes and returns the next ticket according to the queue's ordering.
- Returns null if the queue is empty.

Important: Polling one queue does not remove the corresponding ticket from the other queue.

---

10. Time Complexity

1. offer() – O(log n)
- Inserts an element and restores heap ordering.

2. poll() – O(log n)
- Removes the head and restores heap ordering.

3. peek() – O(1)
- Returns the head without removing it.

4. isEmpty() – O(1)
- Checks whether the queue is empty.

5. size() – O(1)
- Returns the number of elements.

---

11. Important Points to Remember

1. Java PriorityQueue is a Min Heap by default.
2. C++ priority_queue is a Max Heap by default.
3. Java uses `offer()` to insert and `poll()` to remove.
4. `peek()` reads the head without removing it.
5. Custom ordering can be defined using Comparator.
6. PriorityQueue does not maintain a fully sorted list internally.
7. Iterating over a PriorityQueue does not guarantee sorted order.
Repeatedly using `poll()` gives elements in priority order.
8. Java PriorityQueue does not allow null elements.
9. Java PriorityQueue is not thread-safe by default.

---

12. Quick Revision – C++ to Java

C++ → Java

- `push()` → `offer()`
- `pop()` → `poll()`
- `top()` → `peek()`
- `empty()` → `isEmpty()`

```
- Default Max Heap → Default Min Heap
```

PriorityQ.java

```
package com.groww.intern.queuedesk;
import java.util.PriorityQueue;
//Build-T4 Req5: Manages two ticket triage queues using different
ordering strategies.
public class PriorityQ {

    //Queue1: Natural ordering using Comparable.
    private final java.util.PriorityQueue<Ticket> naturalOrderQueue =
new java.util.PriorityQueue<>(); //Creates a priority queue that orders
tickets based on their natural ordering defined in the Ticket class, ie
Our natural ordering: higher priority first, then older createdAt
first.

    //Queue2: Custom ordering using Comparator.
    private final java.util.PriorityQueue<Ticket> customOrderQueue =
new
java.util.PriorityQueue<>(TicketComparators.BY_ASSIGNEE_THEN_PRIORITY);
//Creates a priority queue that orders tickets based on a custom
comparator(BY_ASSIGNEE_THEN_PRIORITY comparator) defined in the
TicketComparators class, ie  Our custom ordering: First sorts by
assignee (alphabetically), then by priority (higher first).

    //Adds the same ticket to both queues.
    public void offer(Ticket ticket){
        naturalOrderQueue.offer(ticket);
        customOrderQueue.offer(ticket);
    }

    //removes and returns the next ticket according to natural
ordering, returns null if the queue is empty.
    public Ticket pollNaturalOrder(){
        return naturalOrderQueue.poll();
    }

    //removes and returns the next ticket according to custom ordering,
returns null if the queue is empty.
```

```
    public Ticket pollCustomOrder(){
        return customOrderQueue.poll();
    }

    //checks whether the natural order queue is empty.
    public boolean isNaturalOrderQueueEmpty(){
        return naturalOrderQueue.isEmpty();
    }

    //checks whether the custom order queue is empty.
    public boolean isCustomOrderQueueEmpty(){
        return customOrderQueue.isEmpty();
    }

}
```

- Write one generic method: static <T extends Ticket> T oldest(List<T> tickets).

TicketUtils.java

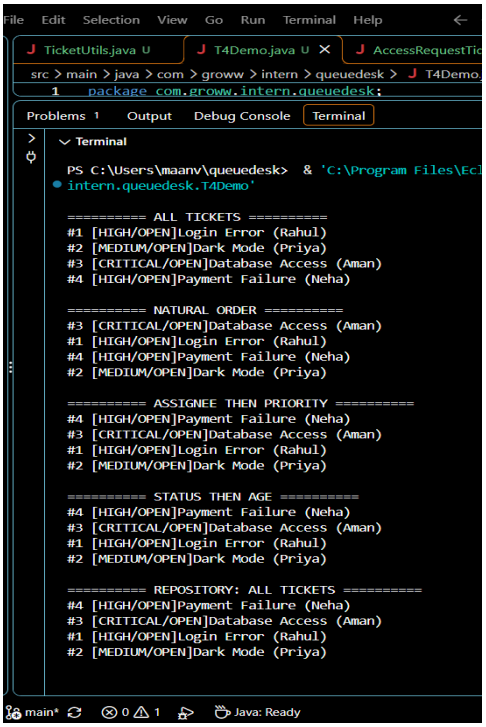
```
package com.groww.intern.queuedesk;
import java.util.List;
//Build-T4 Req6: Utility class containing generic methods for working
with tickets.
public class TicketUtils {
    // Generic method = oldest(List<T> tickets) method to find the
oldest ticket in a list of tickets.
    public static <T extends Ticket> T oldest(List<T> tickets){
//List<T> means the method accepts a list containing objects of type T.
        if(tickets == null || tickets.isEmpty()){ //there is no oldest
ticket if list has no elements.
            return null;
        }
```

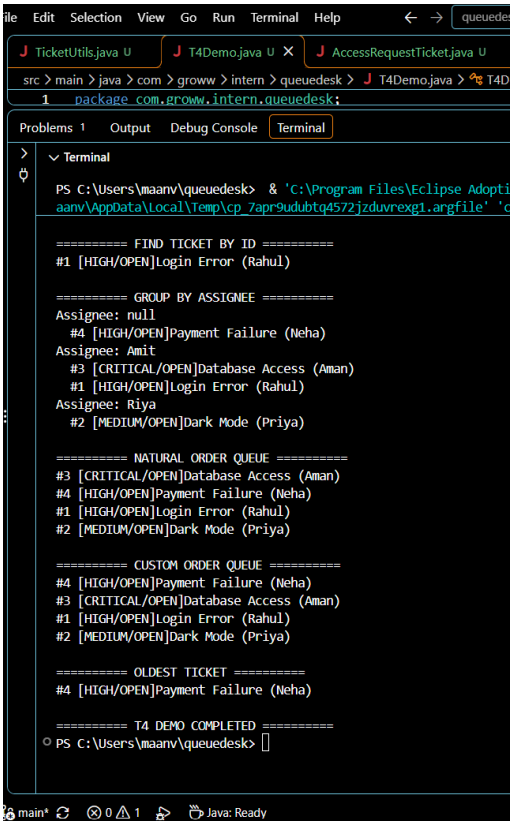
```
        T oldestTicket = tickets.get(0); //Initially assume the first
ticket is the oldest.
        for(T ticket : tickets){ //Iterates through each ticket in the
list.

if(ticket.getCreatedAt().isBefore(oldestTicket.getCreatedAt())){
//isBefore() Checks if the current ticket was created before the oldest
ticket found so far.
                oldestTicket = ticket; //If so, updates the oldest
ticket to the current ticket.
            }
        }
        return oldestTicket; //Returns the oldest ticket found in the
list(ie return the ticket with the earliest creation time).
    }

}
```

T4Demo.java result





BUILD-T5

- Build: loading, saving, reporting
  - InvalidTicketDataException extends Exception (checked) — thrown while parsing a bad line of tickets.txt: wrong field count, an unparseable Priority, etc. Include the offending line number in the message.
  - TicketNotFoundException extends RuntimeException (unchecked) — thrown by Repository.findById(int) when nothing matches. Callers aren't forced to catch it, but the console app should catch it once, at the top, for a clean error message instead of a stack trace.

```
## 1. Exception Handling in Java

An **exception** is an event that interrupts the normal flow of a
program.
For example:
- A file does not exist.
- A line in `tickets.txt` contains an invalid priority.
- A requested ticket ID cannot be found.

Java uses exceptions so that errors can be handled separately from the
normal program logic.

## `try`, `catch`, and `finally`

- `try`: contains code that might cause an exception.
- `catch`: handles an exception thrown from the associated `try` block.
- `finally`: runs whether an exception occurs or not (with a few
exceptional JVM termination cases). It is commonly used for cleanup
when resources are not managed by try-with-resources.

``` java
try{
    int number = Integer.parseInt("abc");
}
catch(NumberFormatException e){
    System.out.println("The value is not a valid number.");
}
finally {
    System.out.println("Parsing attempt finished.");
}
```
```

```

### Checked Exceptions vs Unchecked Exceptions

Java exceptions are commonly discussed in two groups:

# Differences Between Checked and Unchecked Exceptions in Java

| Feature | Checked Exception | Unchecked Exception |
| --- | --- | --- |
| Compiler checking | Checked at compile time. | Not checked by the compiler. |
| Handling requirement | Must be handled using `try-catch` or declared using `throws`. | Handling or declaring is optional. |
| Inheritance | Subclass of `Exception`, excluding `RuntimeException`. | Subclass of `RuntimeException`. |
| Common causes | External or recoverable issues. | Programming mistakes, invalid arguments or states. |
| Examples | `IOException`, `SQLException`, `ClassNotFoundException` | `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException` |

## Checked exceptions

- Checked by the compiler.
- A method must either handle the exception with `try-catch` or declare it using `throws`.
- Often represent conditions the caller may reasonably be expected to handle.
-   Example: `IOException`.

``` java
public void readFile() throws IOException {
    // File-reading code that may throw IOException.
}
```

If a method declares `throws IOException`, it is telling the caller that this operation may fail with an I/O-related problem.
```

Unchecked exceptions

- Subclasses of `RuntimeException` (and `Error`, which is a separate category).
- The compiler does not require callers to catch or declare them.
- Often represent invalid arguments, invalid state, or programming mistakes.
- Example: `IllegalArgumentException`, `NullPointerException`.

``` java
public Ticket findTicket(int id) {
    // May throw an unchecked exception if the ticket is not found.
    throw new IllegalArgumentException("Ticket not found: " + id);
}
```

2. Custom Exceptions

A custom exception is an exception class created by the developer to represent application-specific errors.

A **custom exception** is an exception class created for an application's own error conditions. It gives the error a meaningful name and can carry extra information.

Custom checked exception

Extend `Exception`:

``` java
public class InvalidTicketDataException extends Exception {

    public InvalidTicketDataException(String message) {
        super(message);
    }
}
```

-   `extends Exception` makes it checked.
-   `super(message)` passes the message to the parent `Exception` class.

- The caller must catch it or declare it with `throws`.

For Build T5, the exception should include the offending line number in its message, so the user can identify the bad line in `tickets.txt`.

Custom unchecked exception

Extend `RuntimeException`:

```java
public class TicketNotFoundException extends RuntimeException {

    public TicketNotFoundException(String message) {
        super(message);
    }
}
```

- `extends RuntimeException` makes it unchecked.
- Callers are not required to catch or declare it.
- It can be used by `Repository.findById(int)` when no ticket matches.

When to use each in this project

- Use `InvalidTicketDataException` for malformed input while loading a file. This is a checked exception because loading code should explicitly account for invalid file data.
- Use `TicketNotFoundException` when a requested ticket ID does not exist. This is unchecked, as specified in Build T5.

Differences Between Custom Checked and Custom Unchecked Exceptions in Java

Feature	Custom Checked Exception	Custom Unchecked Exception
Base Class to Extend	`java.lang.Exception`	`java.lang.RuntimeException`
When to Use	For predictable but unavoidable external errors that the caller must recover from.	For business logic violations, invalid program states, or errors caused by bad arguments.
Compiler Enforcement	Forces the calling method to handle it via `try-catch` or declare it via `throws`.	Does not force the caller to handle or declare it.
Example Scenario	Insufficient funds during a bank transfer (caller must decide to retry or notify the user).	Invalid age format passed into a user profile validator (indicates a logic or validation error).

````
## Code Templates

### Custom Checked Exception
```java
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

### Custom Unchecked Exception
```java
public class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```


---
_

## 3. `throws` vs `throw`

These keywords have different purposes:
````

- `throw` actually throws an exception object.
- `throws` declares in a method signature that the method may pass an exception to its caller.

```java
// 'throws' declares the possible exception.
public void loadTickets() throws InvalidTicketDataException {
    // 'throw' throws a specific exception object.
    throw new InvalidTicketDataException("Invalid ticket data on line 4");
}
```

Remember: - `throw` = perform the throw. - `throws` = declare the possibility in the method signature.

Differences Between throw and throws Keywords in Java

Feature	`throw`	`throws`
Purpose	Explicitly throws an exception.	Declares that a method may throw an exception.
Location	Inside a method or block.	In the method signature.
Followed by	An exception object.	Exception class names.
Example	`throw new IOException();`	`void load() throws IOException`

Remember: `throw` = actually throw an exception; `throws` = declare a possible exception.

---

4. Try-with-resources

The try-with-resources statement automatically closes resources when the `try` block finishes, whether it completes normally or exits abruptly due to an exception.

Key Characteristics

* **Automatic Resource Management:** Eliminates the need for explicit `finally` blocks solely dedicated to closing resources.
* **Requirements:** Any resource declared within the parentheses must implement the `java.lang.AutoCloseable` or `java.io.Closeable` interface.
* **Scope:** The declared resources are scoped strictly to the `try` block.

Common Use Cases

* **File Streams:** Managing raw byte or character flows like `FileInputStream` or `FileOutputStream`.
* **Buffered I/O:** Reading and writing text efficiently using `BufferedReader` and `BufferedWriter`.
* **Database Connections:** Ensuring `Connection`, `Statement`, and `ResultSet` objects are released to the connection pool.

Practical Examples

Reading a File
```java
try (BufferedReader reader = Files.newBufferedReader(path)) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}
```

Writing a File
```java
try (BufferedWriter writer = Files.newBufferedWriter(path)) {
    writer.write("Ticket report");
    writer.newLine();
}
```

System Implementation (QueueDesk Build T5)

```
* **Ticket Loading:** The `TicketFileLoader.load(Path file)` component reads `tickets.txt` line by line using a `BufferedReader` enclosed in a try-with-resources block.
* **Report Generation:** The report generation component writes output metrics using a `BufferedWriter` inside a try-with-resources block.
* **Outcome:** Prevents file descriptor leaks by ensuring reader and writer handles are reliably closed under all execution paths.

---

## 5. String Handling

A Java `String` represents text.

### Strings are immutable

**Immutable** means that once a `String` object is created, its contents cannot be changed. String methods that appear to modify text return a new `String`.

``` java
String title = "  Login Error  ";

String cleanedTitle = title.trim();

System.out.println(title);        // Original string remains unchanged.
System.out.println(cleanedTitle); // Trimmed string.
```

Useful String methods

- `trim()` --- removes leading and trailing whitespace characters covered by `trim()`.
- `isBlank()` --- returns `true` if the string is empty or contains only whitespace.
- `isEmpty()` --- returns `true` only if the string has zero characters.
```

- `split("\\|")` --- splits a string around the pipe (`|`) delimiter. The backslash is escaped because the argument is a Java string and the pipe has a special meaning in regular expressions.
- `substring(begin, end)` --- returns part of a string. The start index is included; the end index is excluded.
- `length()` --- returns the number of characters in the string.

Example: splitting a pipe-delimited line

Suppose a file line looks like this:

``` text
101|Login Error|HIGH|OPEN|Rahul
```

Split it using:

``` java
String[] fields = line.split("\\|");

for (String field : fields) {
    System.out.println(field.trim());
}
```

The fields are separated at each pipe character. Calling `trim()` removes surrounding whitespace from each field.

Safe substring / truncation

Do not assume that a string is long enough for a requested substring. Check its length first.

``` java
String description = ticket.getDescription();

int maxLength = 40;

String displayText;
```

```
if (description.length() > maxLength) {
    displayText = description.substring(0, maxLength) + "...";
} else {
    displayText = description;
}
```

-   `substring(0, maxLength)` takes characters from index `0` up to, but
    not including, `maxLength`.
-   The length check prevents an invalid substring range.
-   This is useful when displaying long ticket descriptions on the
    triage board.

Build T5 reminder: Revisit the T-01 setters and use `trim()`, blank checks, and safe `substring()` truncation for long descriptions.

---

6. StringBuilder

`StringBuilder` is a mutable sequence of characters. Unlike `String`, its contents can be changed without creating a new string for every append operation.

It is useful when constructing text in multiple steps, especially inside
loops.

Example using String concatenation

```java
String report = "";

for (Ticket ticket : tickets) {
    report = report + ticket + "\n";
}
```

This repeatedly creates new `String` objects as the report grows.

Example using StringBuilder

```java
StringBuilder report = new StringBuilder();

for (Ticket ticket : tickets) {
    report.append(ticket);
    report.append("\n");
}

String finalReport = report.toString();
```

- `new StringBuilder()` creates a mutable text builder.
- `append()` adds text to the end.
- `toString()` converts the completed content into a `String`.

Useful StringBuilder methods

- `append(value)` --- adds text to the end.
- `append('\n')` --- adds a newline.
- `toString()` --- returns the final text as a `String`.
- `length()` --- returns the current number of characters.

QueueDesk use

Build T5's report generator uses `StringBuilder` to assemble an end-of-day summary containing: - Ticket counts grouped by status. - Ticket counts grouped by priority. - The oldest open ticket. - Every overdue `CRITICAL` ticket.

After assembling the report, convert it to a string if needed, then write it using a `BufferedWriter`.

---

7. Quick revision

```
1.  **Checked exception:** Compiler requires handling or declaration.
    Example: `IOException`, `InvalidTicketDataException`.
2.  **Unchecked exception:** Compiler does not require handling or
    declaration. Example: `NullPointerException`,
    `TicketNotFoundException`.
3.  **Custom exception:** An application-defined exception class.
4.  **`throw`:** Throws an exception object.
5.  **`throws`:** Declares possible exceptions in a method signature.
6.  **Try-with-resources:** Automatically closes resources such as
    readers and writers.
7.  **`trim()`:** Removes leading and trailing whitespace covered by
the
    method.
8.  **`isBlank()`:** Checks whether a string is empty or contains only
    whitespace.
9.  **`split("\\|")`:** Splits a pipe-delimited line.
10. **`substring(start, end)`:** Extracts part of a string; `end` is
    excluded.
11. **`String`:** Immutable text.
12. **`StringBuilder`:** Mutable text builder, useful for assembling
    reports.
```

Added InvalidTicketDataException

```
package com.groww.intern.queuedesk;
/*
 * Build-T5 Req1: InvalidTicketDataException extends Exception.
Invalid file data
 * This exception will be thrown when a line in ticket.txt contains
invalid ticket data, for eg:
 * 1. wrong field count - Wrong number of fields.
 * 2. an unparseable Priority - Invalid priority such as URGENT when it
is not a valid enum value.
 * 3. Invalid ticket information while parsing.
 *
 * Custom Checked Exception used when a ticket record contains invalid
data while reading tickets.txt.
 * Extending exception makes this a checked exception.
 * Therefore, the caller must either handle it using try-catch block or
declare it using the throws keyword.
```

```
 */
public class InvalidTicketDataException extends Exception {
    public InvalidTicketDataException(String message){ //a constructor
that accepts an error message as input i.e. String message.
        super(message); //calls the parent exception class constructor
and passes the error message to it. This allows us to retrieve the
message later using getMessage().
    }
}
```

And added TicketNotFoundException

```
package com.groww.intern.queuedesk;
/*
Build-T5 Req2: TicketNotFoundException extends RunTimeException.
Ticket ID not found
Custom Unchecked Exception used when a requested ticket cannot be found
in the repository.
Extending RuntimeException makes this an unchecked exception.
Therefore, the caller is not required to handle it using try-catch
block or declare it using the throws keyword.
*/
public class TicketNotFoundException extends RuntimeException {
    public TicketNotFoundException(String message){ //a constructor
that accepts an error message as input ie String message.
        super(message); // calls the parent RuntimeException class
constructor and passes the error message to it. This allows us to
retrieve the message later using getMessage().
    }
}
```

Integrated **TicketNotFoundException** into **Repository.java.**

TicketNotFoundException extends **RuntimeException** — thrown by Repository.findById(int) when nothing matches.

if the requested ticket doesn't exist, the method silently returns null.

Instead, we want it to throw our custom exception with a meaningful message.

In repository.java

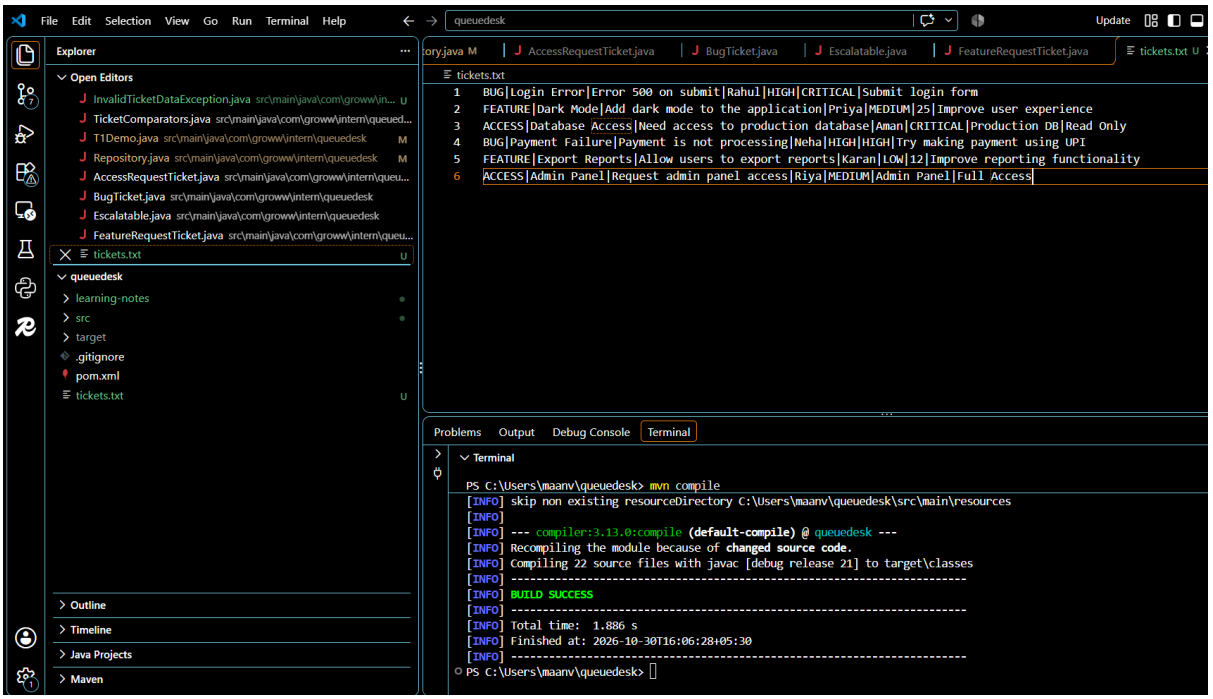
```
//return null; //If no ticket with the given ID is found, returns null.

        //Build T5 Req2: Throw TicketNotFoundException if the ticket
with the given ID is not found.So, throw a custom unchecked exception
instead of returning null.

        throw new TicketNotFoundException("Ticket with ID" + id + "was
not found.");
```

- TicketFileLoader.load(Path file) — reads tickets.txt (pipe-delimited) line by line with try-with-resources over a BufferedReader, using String.split("\\|") and trim() on each field, wrapping any parse failure into InvalidTicketDataException.

tickets.txt



Field order

- BUG: Type | Title | Description | Requester | Priority | Severity | Steps
- FEATURE: Type | Title | Description | Requester | Priority | Votes | Business Justification
- ACCESS: Type | Title | Description | Requester | Priority | System Name | Access Level

6 sample tickets - 2 of each type

- TicketFileLoader.load(Path file) — reads tickets.txt (pipe-delimited) line by line with try-with-resources over a BufferedReader, using String.split("\\|") and trim() on each field, wrapping any parse failure into InvalidTicketDataException.

Added TicketFileLoader.java

- A report generator that assembles an end-of-day summary — counts per status and priority, the oldest open ticket, every overdue CRITICAL ticket — using StringBuilder, then writes it with try-with-resources over a BufferedWriter.

Added **TicketReportGeneration.java**

We used `StringBuilder` because the end-of-day report is dynamically generated by combining multiple sections, including ticket counts by status, ticket counts by priority, the oldest OPEN ticket, and all overdue CRITICAL tickets. Since `String` is immutable, repeated concatenation can create multiple temporary String objects. `StringBuilder` is mutable, allowing us to append multiple pieces of text to the same object efficiently. Once the complete report is assembled, we use `toString()` to convert it into a normal String and write it to the output file using `BufferedWriter`.

- Mutable: Its content can be modified without creating a new String for every append operation.
- Performance: More efficient for repeated text concatenation than repeatedly creating new Strings.
- append(): Adds report headings, ticket counts, and ticket details.
- toString(): Converts the completed report into a normal String.
- BufferedWriter: Writes the generated report into a file.
- Why not StringBuffer? StringBuffer synchronizes its methods, whereas our report is built locally by one thread. Therefore, StringBuilder is more suitable and avoids unnecessary synchronization overhead.

- Revisit the T-01 setters: use trim(), blank checks, and safe substring() truncation for long descriptions on the triage board display.

```
//Build-T5 Req6: Returns a shortened description for displaying on the
triage board. If the description is longer than maxLength, it truncates
```

```
it and appends "...". If it's shorter or equal, it returns the full
description.
    // Returns a shortened description for displaying on the triage
board.
    public String getDescriptionPreview(int maxLength) {
        // Maximum preview length cannot be negative.
        if (maxLength < 0) {
            throw new IllegalArgumentException("Maximum length cannot
be negative");
        }

        // If description is already within the limit, return it as it
is.
        if (description.length() <= maxLength) {
            return description;
        }

        // substring(0, maxLength) extracts characters from index 0
        // up to maxLength (excluding maxLength).
        // We append "..." to indicate that the description has been
shortened.
        return description.substring(0, Math.min(maxLength,
description.length())) + "...";
    }
```

T5Demo.java

```
package com.queuedesk;

// Used to represent file paths.
import java.nio.file.Path;

// List stores multiple ticket objects.
import java.util.List;



// Build-T5: Demonstrates loading, exception handling, and report
generation.
public class T5Demo {

    public static void main(String[] args) {
```

```
        /*
         * STEP 1: LOAD TICKETS FROM tickets.txt
         *
         * TicketFileLoader.load() reads the file line by line.
         * It returns a List<String[]> containing the fields of each
line.
         */

        Path inputFile = Path.of("tickets.txt");

        try{
            // Load the ticket records from the input file.
            List<String[]> records = TicketFileLoader.load(inputFile);

            // Display how many records were loaded.
            System.out.println("Tickets loaded successfully: " +
records.size());

        }
        catch (InvalidTicketDataException e) {
            // Handles invalid data found while parsing the file.
            // getMessage() displays the error message, including the
line number.
            System.out.println("Invalid ticket data: " +
e.getMessage());

        }
        catch (java.io.IOException e) {
            // Handles file-related errors such as missing files or
access issues.
            System.out.println("Unable to read tickets file: " +
e.getMessage());
        }


        /*
         * STEP 2: DEMONSTRATE TICKETNOTFOUNDEXCEPTION
         *
         * Repository stores Ticket objects.
```

```
         * findById() throws TicketNotFoundException if the requested
ID
         * does not exist in the repository.
         */

        Repository<Ticket> repository = new Repository<>();

        // Add a ticket to the repository.
        Ticket ticket = new FeatureRequestTicket(
                "Login improvement",
                "Improve the login experience",
                "Aman",
                Priority.HIGH,
                5,
                "Improve user experience"
        );

        repository.add(ticket);

        try {

            // Search for a ticket that does not exist.
            repository.findById(9999);

        }
        catch (TicketNotFoundException e) {

            // Catch the unchecked exception and display a clean
message.
            System.out.println("Repository error: " + e.getMessage());
        }


        /*
         * STEP 3: GENERATE THE END-OF-DAY REPORT
         *
         * TicketReportGenerator generates a summary containing:
         * 1. Counts by status.
         * 2. Counts by priority.
         * 3. The oldest OPEN ticket.
```

```
         * 4. Every overdue CRITICAL ticket.
         *
         * The report is saved using BufferedWriter.
         */

        // Specify the location of the output report.
        Path outputFile = Path.of("ticket_report.txt");

        try {

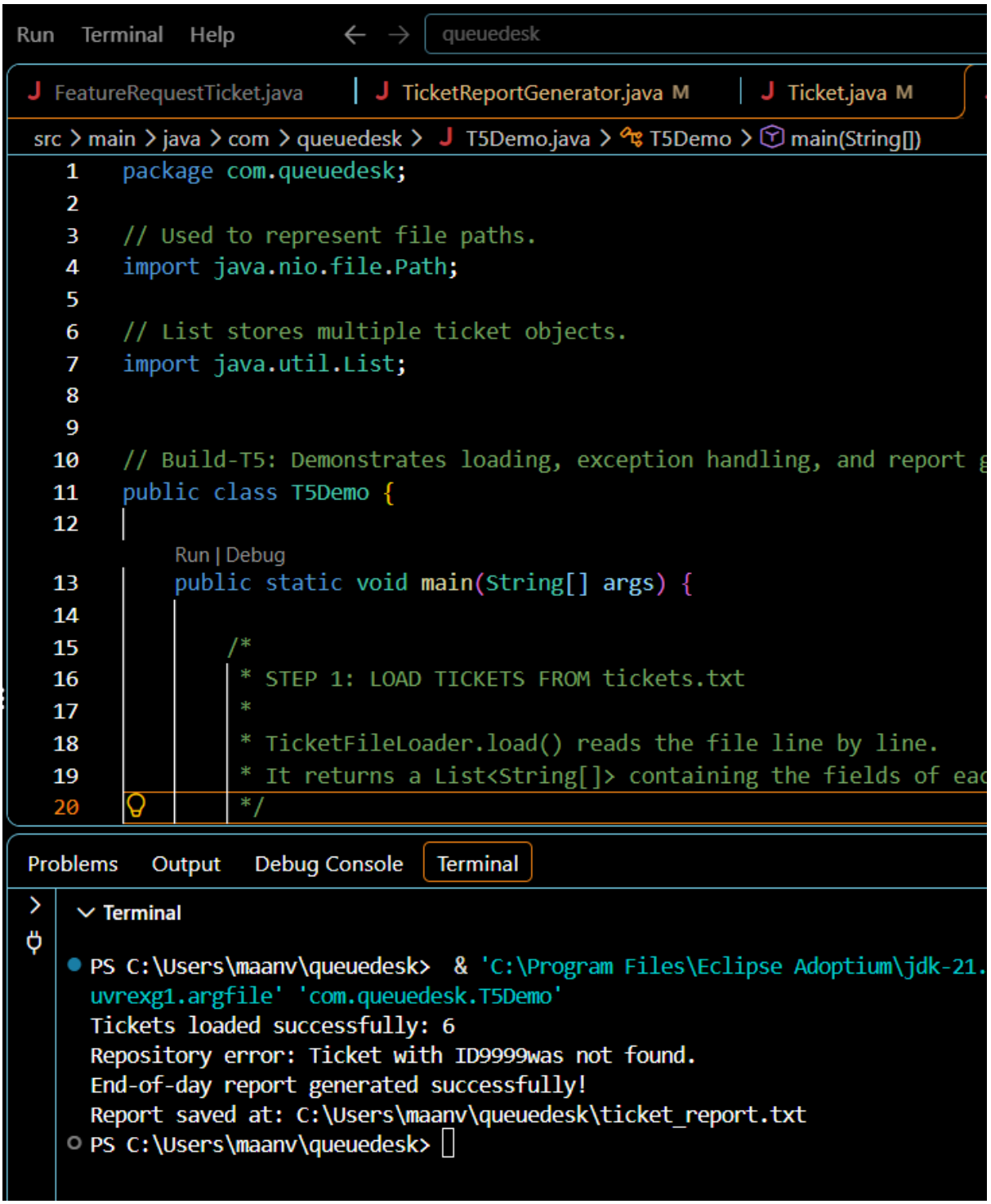
            // Generate and save the report.
            TicketReportGenerator.generateReport(
                    repository.all(),
                    outputFile
            );

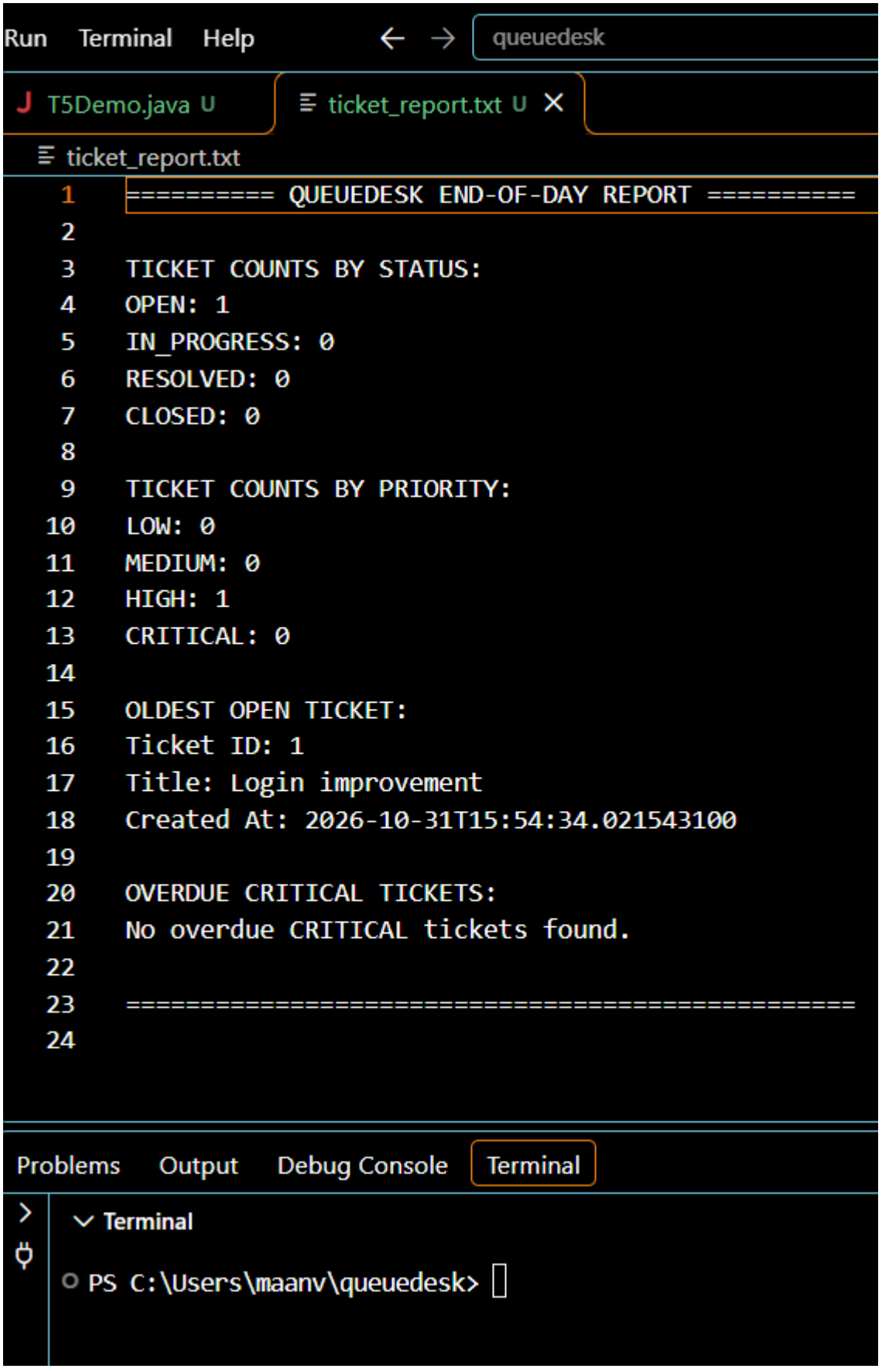
            System.out.println("End-of-day report generated
successfully!");
            System.out.println("Report saved at: " +
outputFile.toAbsolutePath());

        }
        catch (java.io.IOException e) {

            // Handle errors that occur while writing the report.
            System.out.println("Unable to generate report: " +
e.getMessage());
        }
    }
}
```

Result:


Run Terminal Help
queuedesk
FeatureRequestTicket.java
TicketReportGenerator.java M
Ticket.java M
src > main > java > com > queuedesk > T5Demo.java > T5Demo > main(String[])
1 package com.queuedesk;
2
3 // Used to represent file paths.
4 import java.nio.file.Path;
5
6 // List stores multiple ticket objects.
7 import java.util.List;
8
9
10 // Build-T5: Demonstrates loading, exception handling, and report g
11 public class T5Demo {
12
Run | Debug
13 public static void main(String[] args) {
14
15 /*
16 * STEP 1: LOAD TICKETS FROM tickets.txt
17 *
18 * TicketFileLoader.load() reads the file line by line.
19 * It returns a List<String[]> containing the fields of eac
20 */
Problems Output Debug Console Terminal
Terminal
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.
uvrexg1.argfile' 'com.queuedesk.T5Demo'
Tickets loaded successfully: 6
Repository error: Ticket with ID9999was not found.
End-of-day report generated successfully!
Report saved at: C:\Users\maanv\queuedesk\ticket_report.txt
PS C:\Users\maanv\queuedesk>


Run Terminal Help
queuedesk
T5Demo.java U
ticket_report.txt U
ticket_report.txt
1 ========== QUEUEDESK END-OF-DAY REPORT ==========
2
3 TICKET COUNTS BY STATUS:
4 OPEN: 1
5 IN_PROGRESS: 0
6 RESOLVED: 0
7 CLOSED: 0
8
9 TICKET COUNTS BY PRIORITY:
10 LOW: 0
11 MEDIUM: 0
12 HIGH: 1
13 CRITICAL: 0
14
15 OLDEST OPEN TICKET:
16 Ticket ID: 1
17 Title: Login improvement
18 Created At: 2026-10-31T15:54:34.021543100
19
20 OVERDUE CRITICAL TICKETS:
21 No overdue CRITICAL tickets found.
22
23 ==
24
Problems Output Debug Console Terminal
Terminal
PS C:\Users\maanv\queuedesk>

Build-T6

- Build: TicketAnalytics
  - Map<Priority, Long> countByPriority(List<Ticket> tickets) — stream().collect(Collectors.groupingBy(Ticket::getPriority, Collectors.counting())).
  - Map<String, List<Ticket>> groupByAssignee(List<Ticket> tickets) — rewrite the T-03 loop version as a one-line Collectors.groupingBy stream.
  - double averageEffortHours(List<Ticket> tickets) — via mapToDouble(Ticket::estimateEffortHours).average(), handling the empty-list case explicitly.
  - double totalEffortHours(List<Ticket> tickets) — the same total from T-05, this time via .stream().reduce(0.0, Double::sum, Double::sum), so the two approaches sit side by side for comparison.
  - Optional<Ticket> oldestOpenTicket(List<Ticket> tickets) — filter to OPEN, then min(Comparator.comparing(Ticket::getCreatedAt)), which already returns Optional<Ticket>.
  - Add Optional<Ticket> Repository<T>.findByIdSafe(int id) alongside the existing T-04 findById that throws — same lookup, two calling conventions. Use it with .map(...), .ifPresentOrElse(...), and .orElseThrow(() → new TicketNotFoundException(id)) in different call sites, matching the convention to the caller's needs.
  - A String tagLine(List<Ticket> tickets) using flatMap over each ticket's tag Set<String>, distinct(), sorted(), and Collectors.joining(", ").

Added **TicketAnalytics.java**
Req1,2,3,4,5,7
Added Req6 i**n Repository.java**

Build T6 – TicketAnalytics: Java Concepts and Notes

Build T6 brings together the Java concepts used in QueueDesk to analyse ticket data. The main focus is Generics, Functional Interfaces, Lambda Expressions, Streams API, and Optional.

The key idea is to process collections of tickets declaratively: instead of writing loops to count, group, filter, sort, and calculate values, we describe the operation using streams.

1. Generics

Generics allow classes, interfaces, and methods to work with different data types while maintaining compile-time type safety.

- A type parameter acts as a placeholder for a data type.

- Common convention: `T` for Type, `E` for Element, `K` for Key, and `V` for Value.
- Generics reduce explicit type casting and help catch type errors during compilation.

Example:

```java
List<Ticket> tickets = new ArrayList<>();
```

- `List` is the generic interface.
- `Ticket` is the type parameter.
- The list is restricted to Ticket objects.

Bounded Generics

Bounded generics restrict which types can be passed as a type argument.

```java
public class Repository<T extends Ticket> {
    // Stores Ticket objects or their subclasses
}
```

- `T extends Ticket` means T must be Ticket or a subclass of Ticket.
- It allows the repository to use methods defined in Ticket.
- Example: `Repository<BugTicket>` or `Repository<FeatureTicket>`.

Wildcards

Wildcards (`?`) represent an unknown type.

Syntax	Meaning
`<?>`	List of any type
`<? extends Ticket>`	Ticket or any subclass of Ticket
`<? super BugTicket>`	BugTicket or one of its supertypes

PECS principle: Producer Extends, Consumer Super

- `? extends Ticket`: use when a collection produces Ticket values for reading.
- `? super BugTicket`: use when a collection consumes BugTicket values for adding.
- `<?>`: use when the exact type is not important.

Functional Interface

- A Functional Interface is an interface with exactly one Abstract Method (SAM - Single Abstract Method).
- It defines one operation that can be implemented using Lambda Expression.
  meant:
  A Functional Interface define what operation is required & a Lambda Expression provides the implementation of how that operation is performed.
- It may also contain default & static methods.
- @FunctionalInterface is optional; but asks the compiler to check that there is exactly 1 abstract method.

Syntax:

① @FunctionalInterface
interface Calculator {
    int calculate (int a, int b);
}

② class Addition implements Calculator {
    @override
    public int calculate (int a, int b) {
        return a + b;
    }
}

Calculator calculator = new Calculator(); ✗
as an interface cannot be instantiated directly

public class Main {
    public static void main (String[] args) {
        Calculator (calculator) = new Addition();
        System.out.println (calculator.calculate (10, 5));
    }
}

obj

Creating an obj of Addition & storing it's reference in a variable called obj, whose type is Calculator (interface).

Output: 15

class Addition implements Calculator
class Subtraction implements Calculator
class multiplication implements Calculator

without Interface reference → Addition obj = new Addition();
new obj is specifically an Addition reference if we want subtraction we need another reference.
Subtraction obj2 = new Subtraction();
i.e. Code becomes dependent on specific classes.

The reference type is remains same. Only the implementation changes.

① Calculator obj = new Addition();
② Calculator obj = new Subtraction();

Q. Why useful in real projects?
Developing a payment application: → UPI payment
→ Card/Credit payment
→ Net Banking payment.

All implement a common Interface:
interface Payment {
    void Pay();
}

Payment p = new UpiPayment();
p.pay();

Payment p = new CardPayment();
p.pay();

① to Achieve Runtime polymorphism
② To Reduce dependency on Specific Implementation classes.
③ to make code flexible & loosely coupled.
④ to easily switch b/w different implementations without changing the reference type.

Lambda Expression : It is a concise way to provide the implementation of the single abstract method (SAM) of a functional interface, without creating a seperate implementation class.

Syntax : (parameters) → expression ;

() → contains parameters (inputs) | → : seperate parameters from the body
expression : contains the Actual implementation (what to do).

① No parameter
() → System.out.println("Hello Java");

② One parameter
name → System.out.println(name);

③ Two parameters
(a,b) → a+b ;

④ Multiple Statements
(a,b) → {
int sum = a+b;
return sum;
};

Same Calculator Eg:

@FunctionalInterface
interface Calculator {
int calculate(int a, int b);
}

public class Main {
public static void main(String[] args) {
Calculator obj = (a,b) → a+b ;
int result = obj.calculate(10,5);
System.out.println(result);
} }

obj holds the lambda implementation
obj.calculate() → calls the method declared in the interface.

```
@FunctionalInterface
interface Printer {
    void print();
}
```

① using implements

```
class MessagePrinter implements Printer {
    @Override
    public void print() {
        System.out.println("Hello Java");
    }
}

public class Main {
    public static void (String[] args) {
        Printer p = new MessagePrinter();
        p.print();
    }
}
```

② using Lambda Expression

Not needed.

```
Printer p = () -> System.out.println("Hello Java");
p.print();
```

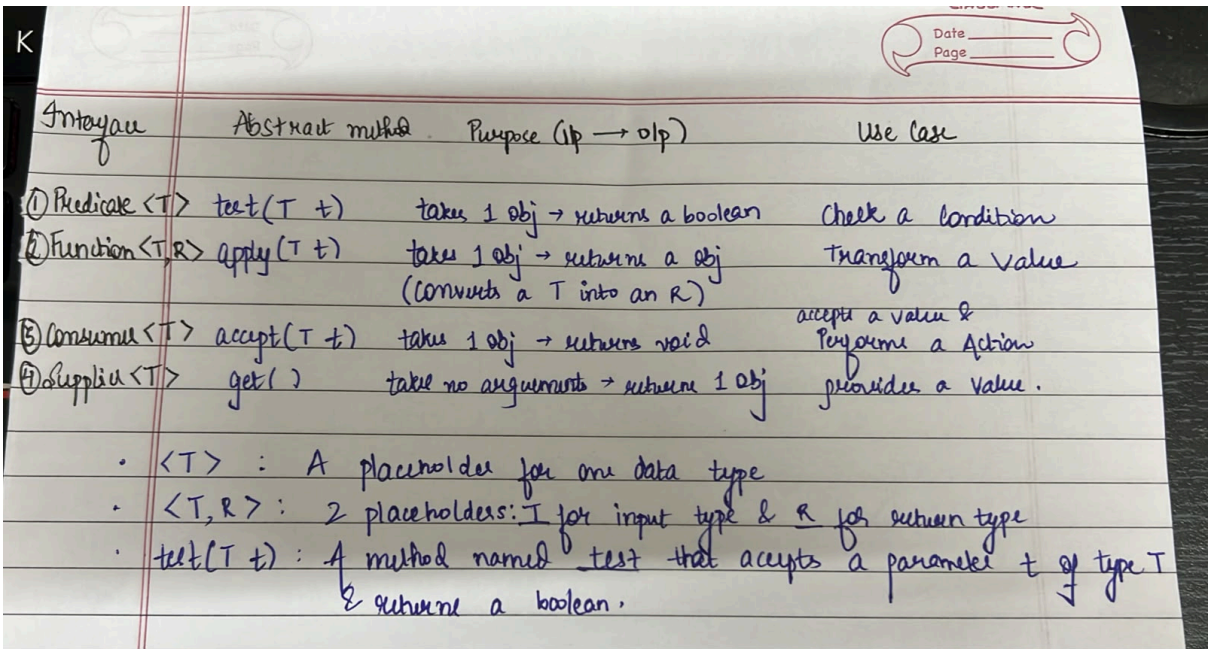
Output: Hello java

* implements require us to make a seperate class to provide the interface method implementation, whereas Lambda Expression provides that implementation directly in a shorter way without creating a seperate named class.

Why needed? ① to Reduce Boilerplate code
② make code cleaner.
③ pass behaviour as an argument or store it in a variable.

Types of Functional Interfaces

Java8 introduced the java.util.function package, which contains Builtin functional Interfaces.



What does <T> mean?

T is a generic type parameter. It acts as a placeholder for a data type.

```
None
Predicate<Integer>
Predicate<String>
Predicate<Ticket>
```

Here, T is replaced by `Integer`, `String`, or `Ticket`, respectively.

What does <T, R> mean?

It represents two generic type parameters:

- T = Input type
- R = Return type

For example:

```
None
Function<String, Integer>
```

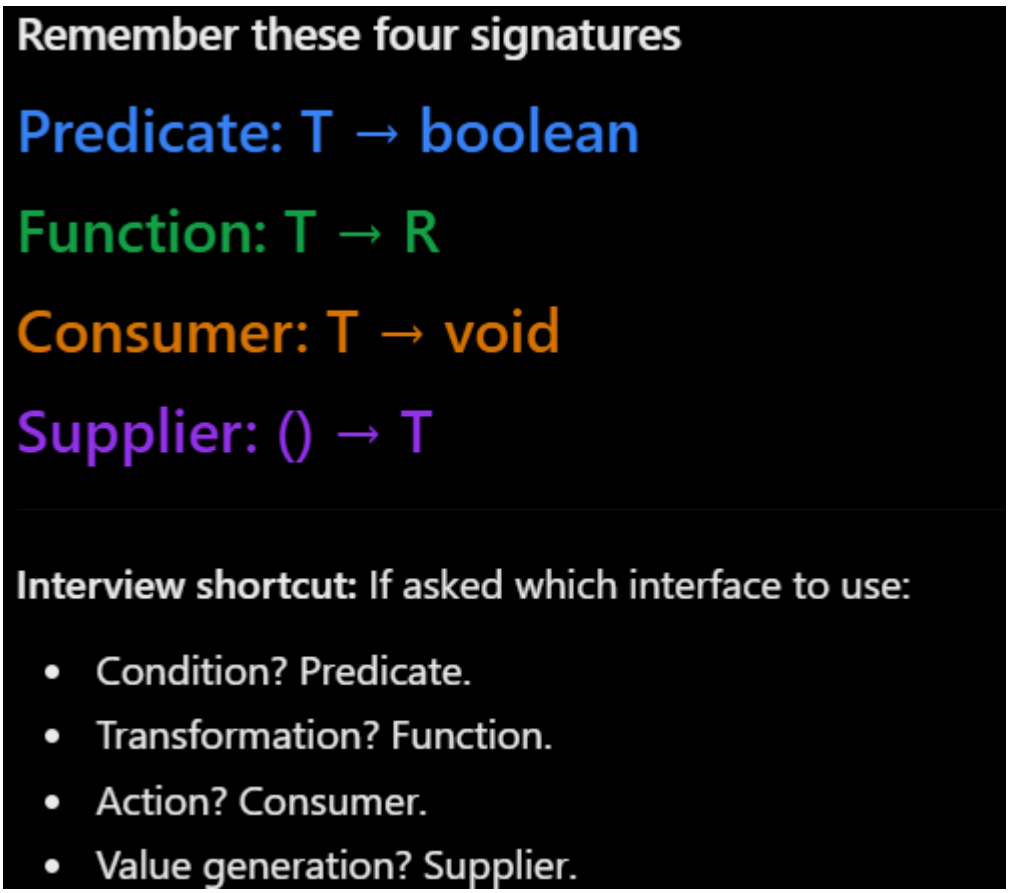
This means the function accepts a `String` and returns an `Integer`.

3. Comparison table – Important for interviews

Interface	Abstract method	Input	Output	Purpose
`Predicate<T>`	`test(T)`	1	boolean	Check condition
`Function<T,R>`	`apply(T)`	1	R	Transform value
`Consumer<T>`	`accept(T)`	1	void	Perform action
`Supplier<T>`	`get()`	0	T	Supply value

Easy way to remember:

- Predicate → Is this condition true?
- Function → Convert this into something else.
- Consumer → Do something with this value.
- Supplier → Give me a value.



1. What is the Streams API?

The Java Streams API, introduced in Java 8, allows us to process data from collections using operations such as filtering, mapping, sorting, grouping and aggregation.

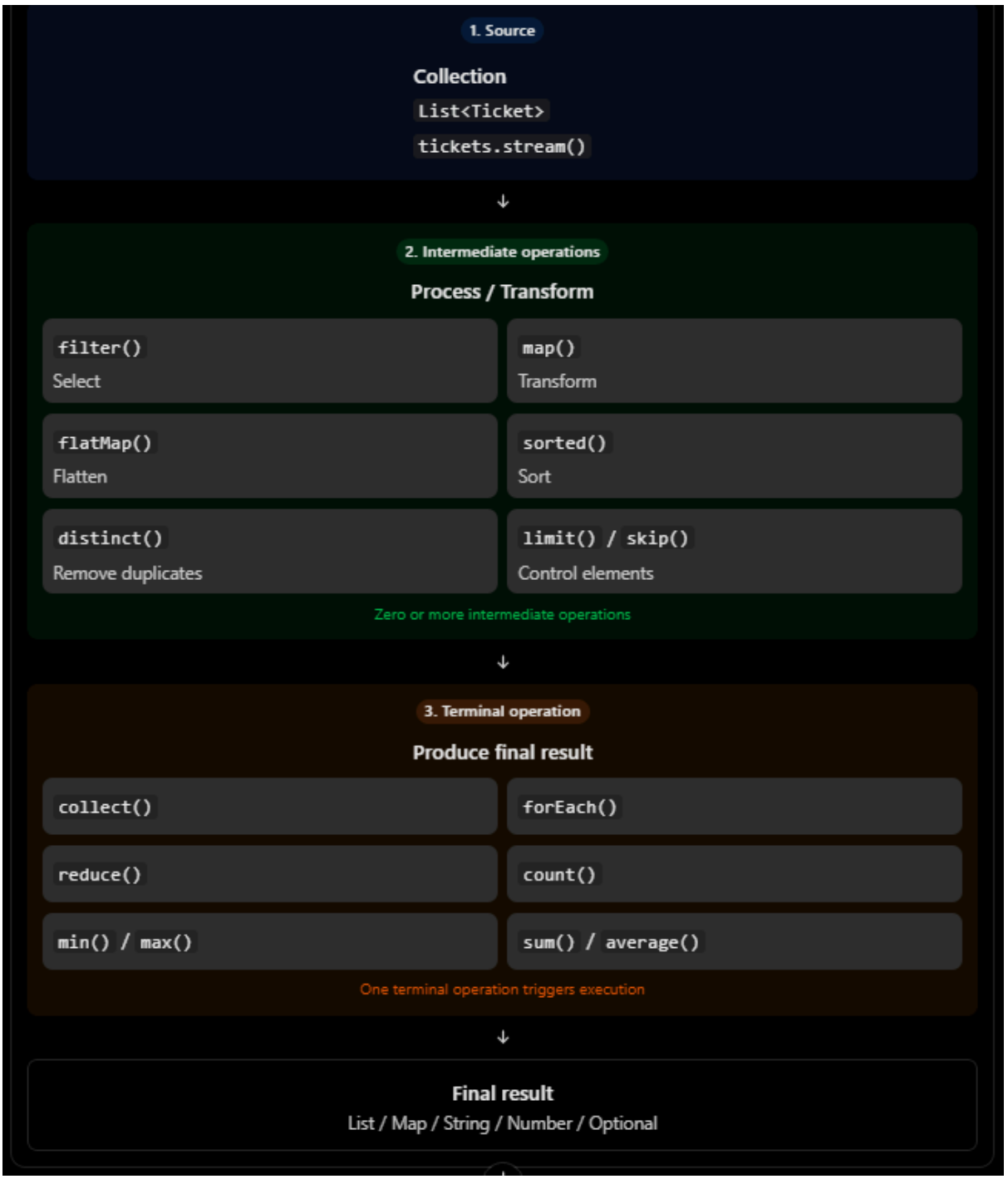
Why do we use it?

- To process collections without writing repetitive loops.
- To filter data based on conditions.
- To transform one type of data into another.
- To perform calculations such as sum, count and average.
- To group and collect data for reports and analytics.

Important: A Stream does not store data. It processes data from a source, such as a List or Set.

2. Stream pipeline – architecture

A Stream pipeline has three stages:



General syntax:

```
collection.stream()
```

```
    .intermediateOperation1()
    .intermediateOperation2()
    .terminalOperation();
```

- Source: Where does the data come from?
- Intermediate: What processing should happen?
- Terminal: What result do we need?

Intermediate operations are generally lazy. They execute when a terminal operation is invoked.

3. Important Stream operations and syntax

Intermediate operations

Operation	Syntax	Purpose
`filter()`	`.filter(x -> condition)`	Select matching elements
`map()`	`.map(x -> transformation)`	Transform each element
`mapToDouble()`	`.mapToDouble(x -> value)`	Convert to DoubleStream
`flatMap()`	`.flatMap(x -> x.stream())`	Flatten nested streams

`distinct()`	`.distinct()`	Remove duplicates
`sorted()`	`.sorted()`	Sort elements
`limit(n)`	`.limit(n)`	Keep first n elements
`skip(n)`	`.skip(n)`	Skip first n elements

Terminal operations

Operation	Syntax	Purpose
`collect()`	`.collect(Collectors.toList())`	Collect into a result container
`forEach()`	`.forEach(x -> action)`	Perform an action
`reduce()`	`.reduce(identity, accumulator)`	Combine values
`count()`	`.count()`	Count elements
`min()`	`.min(comparator)`	Find minimum
`max()`	`.max(comparator)`	Find maximum

`findFirst()`	`.findFirst()`	Find first element
`anyMatch()`	`.anyMatch(condition)`	Check if any element matches
`allMatch()`	`.allMatch(condition)`	Check if all elements match

4. Collectors and collect()

This is one of the most important concepts in your Build T6.

- `collect()` is a terminal operation.
- `Collectors` is a utility class that provides predefined ways to collect stream elements.

Think of it as:

- `collect()` = perform the collection.
- `Collectors` = specify how to collect.

Common Collectors methods

Method	Purpose
`toList()`	Collect into a List
`toSet()`	Collect into a Set

`groupingBy()`	Group elements based on a property
`counting()`	Count elements
`joining()`	Join strings using a delimiter
`summingDouble()`	Calculate sum of double values
`averagingDouble()`	Calculate average of double values

Example:

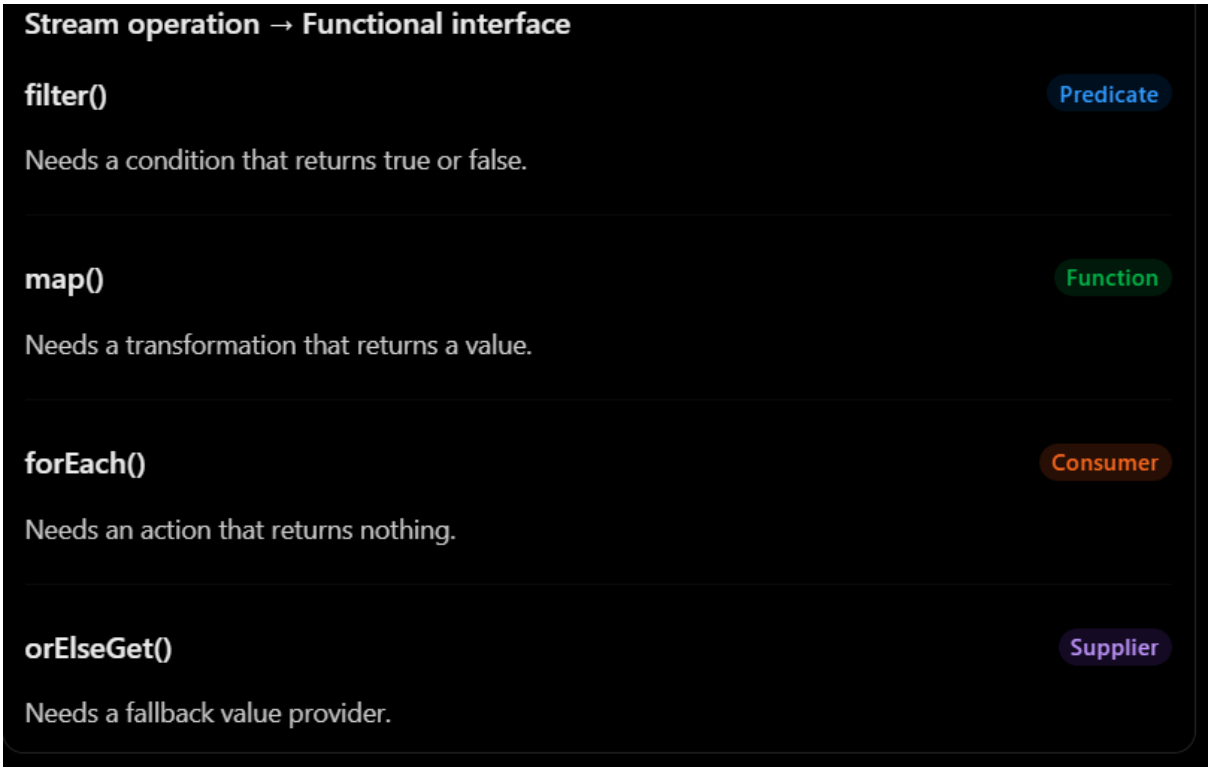
```
None
tickets.stream()
    .collect(Collectors.groupingBy(
        Ticket::getPriority,
        Collectors.counting()
    ));
```

This groups tickets by priority and counts how many tickets belong to each group.

Result:

```
None
HIGH   → 2
MEDIUM → 3
LOW    → 1
```

Here, the return type is `Map<Priority, Long>`.



Each Stream operation expects a particular type of functional interface depending on what it needs to do with every element.

- `filter()` needs a yes/no decision → Predicate.
- `map()` needs a transformation → Function.
- `forEach()` needs an action → Consumer.

Let's understand this using your QueueDesk project.

1. Why does `filter()` take a Predicate?

Purpose: Select elements based on a condition.

Suppose your QueueDesk has 5 tickets, but you want only the OPEN tickets.

```
None
tickets.stream()

    .filter(ticket -> ticket.getStatus() == Status.OPEN)
```

```
    .toList();
```

What happens internally?

1. `filter()` takes one ticket at a time.
2. It checks whether the ticket's status is OPEN.
3. The condition returns either `true` or `false`.
4. If `true`, the ticket stays in the resulting stream.
5. If `false`, the ticket is discarded from the resulting stream.

Why Predicate?

Because Predicate's abstract method is:

```
None
boolean test(T t);
```

It accepts an object and returns a boolean. That is exactly what `filter()` needs.

Interview answer: "`filter()` takes a Predicate because it needs a boolean condition to decide whether each element should be included in the resulting stream."

2. Why does `map()` take a Function?

Purpose: Transform each element into another value.

Suppose you have a list of Ticket objects, but you only want their priorities.

```
None
tickets.stream()

    .map(Ticket::getPriority)

    .toList();
```

What happens?

1. `map()` takes one Ticket object.
2. It calls `getPriority()` on that ticket.
3. It returns a Priority value.
4. It produces a new stream containing Priority values instead of Ticket objects.

Why Function?

Because Function's abstract method is:

```
None
R apply(T t);
```

- `T` = Input type (Ticket).
- `R` = Return type (Priority).

It accepts one type and returns potentially another type.

Interview answer: "`map()` takes a Function because it transforms each stream element into another value. In QueueDesk, it can transform a Ticket into its Priority or estimated effort."

3. Why does `forEach()` take a Consumer?

Purpose: Perform an action on each element.

Suppose you want to print all tickets.

```
None
tickets.stream()

    .forEach(ticket -> System.out.println(ticket));
```

What happens?

1. `forEach()` takes one Ticket.
2. It passes the ticket to the lambda expression.
3. The lambda prints the ticket.
4. It repeats for every ticket.

5. It does not produce a new collection or return a value.

Why Consumer?

Because Consumer's abstract method is:

```
None
void accept(T t);
```

It accepts an object but returns nothing.

Interview answer: "`forEach()` takes a Consumer because it performs an action on each element without returning a result. In QueueDesk, we can use it to print ticket details or log information."

4. Compare all three

Operation	Functional interface	What it expects	What it does
`filter()`	Predicate	T → boolean	Selects elements
`map()`	Function	T → R	Transforms elements
`forEach()`	Consumer	T → void	Performs an action

5. One QueueDesk example combining all three

Suppose you want to print the titles of all OPEN tickets.

None

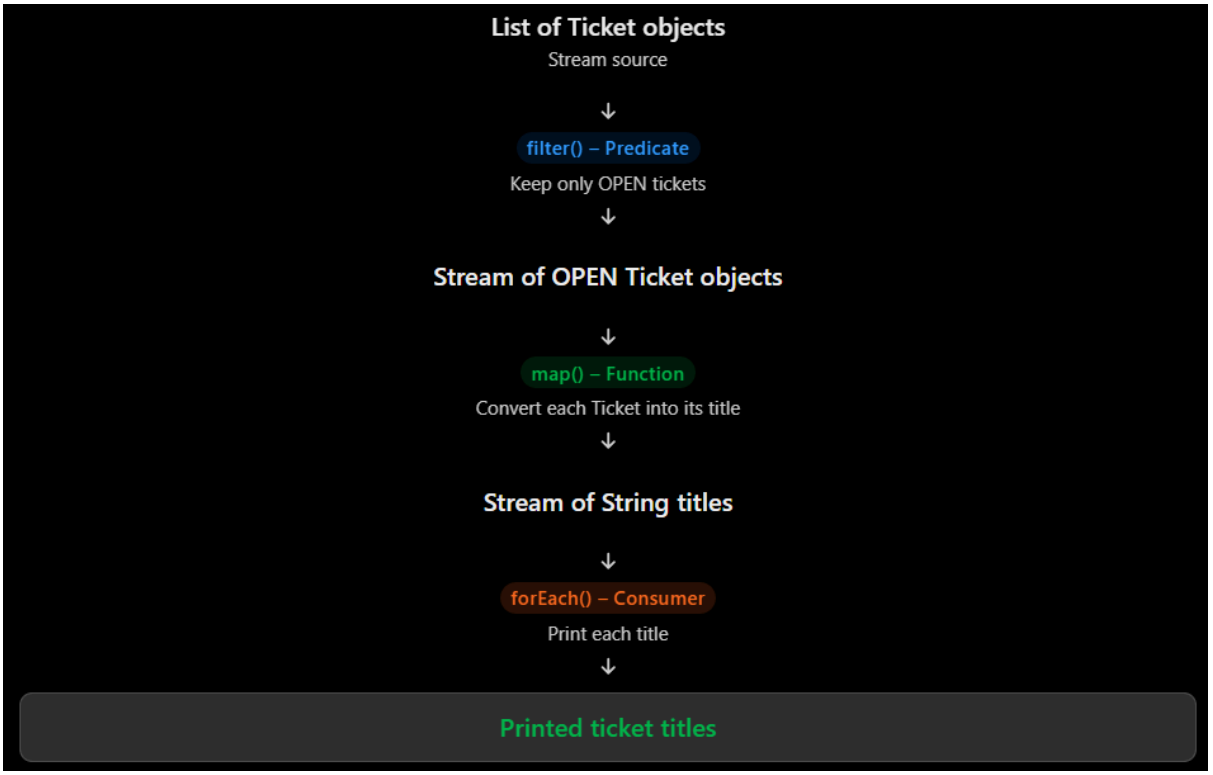
```
tickets.stream()


    // 1. Select only OPEN tickets
    .filter(ticket ->
        ticket.getStatus() == Status.OPEN)


    // 2. Convert Ticket objects into String titles
    .map(Ticket::getTitle)


    // 3. Print each title
    .forEach(System.out::println);
```

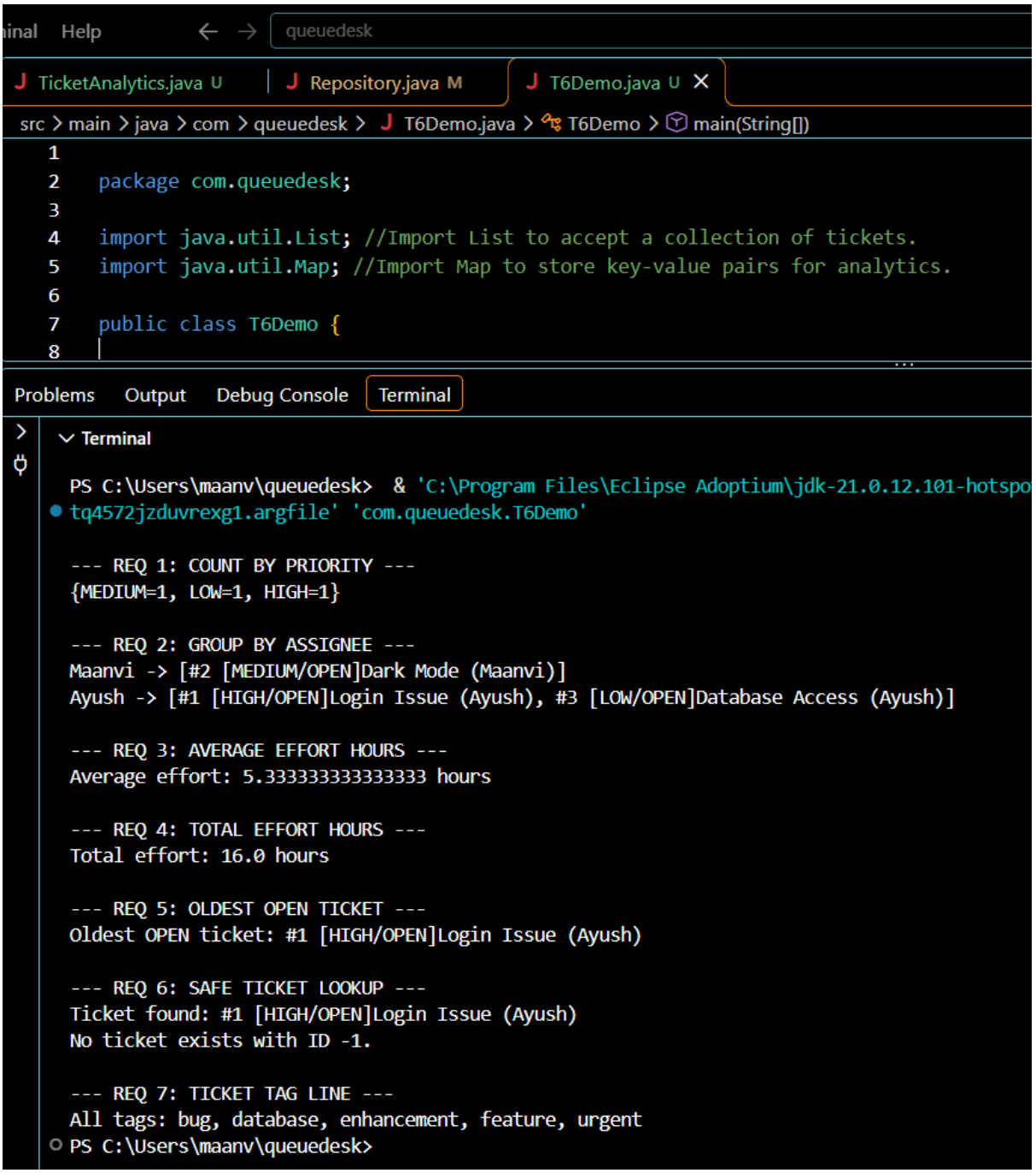
How the data flows



`Stream<Ticket>` → `Stream<Ticket>` → `Stream<String>` → printed output.

- `filter()` keeps the same type but reduces the number of elements.
- `map()` can change the element type.
- `forEach()` consumes the elements to perform an action.

T6Demo.java


inal Help
queuedesk
TicketAnalytics.java U
Repository.java M
T6Demo.java U
src > main > java > com > queuedesk > T6Demo.java > T6Demo > main(String[])
1
2 package com.queuedesk;
3
4 import java.util.List; //Import List to accept a collection of tickets.
5 import java.util.Map; //Import Map to store key-value pairs for analytics.
6
7 public class T6Demo {
8
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspo
tq4572jzduvrexg1.argfile' 'com.queuedesk.T6Demo'
--- REQ 1: COUNT BY PRIORITY ---
{MEDIUM=1, LOW=1, HIGH=1}
--- REQ 2: GROUP BY ASSIGNEE ---
Maanvi -> [#2 [MEDIUM/OPEN]Dark Mode (Maanvi)]
Ayush -> [#1 [HIGH/OPEN]Login Issue (Ayush), #3 [LOW/OPEN]Database Access (Ayush)]
--- REQ 3: AVERAGE EFFORT HOURS ---
Average effort: 5.333333333333333 hours
--- REQ 4: TOTAL EFFORT HOURS ---
Total effort: 16.0 hours
--- REQ 5: OLDEST OPEN TICKET ---
Oldest OPEN ticket: #1 [HIGH/OPEN]Login Issue (Ayush)
--- REQ 6: SAFE TICKET LOOKUP ---
Ticket found: #1 [HIGH/OPEN]Login Issue (Ayush)
No ticket exists with ID -1.
--- REQ 7: TICKET TAG LINE ---
All tags: bug, database, enhancement, feature, urgent
PS C:\Users\maanv\queuedesk>

Build-T7

- Build: tickets.json
  - Add @JsonTypeInfo(use = JsonTypeInfo.Id.NAME, include = JsonTypeInfo.As.PROPERTY, property = "type") and @JsonSubTypes to Ticket, naming each subtype ("bug", "feature", "access").
  - Build the mapper once, in one place: new ObjectMapper().registerModule(new JavaTimeModule()).disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS).enable(SerializationFeature.INDENT_OUTPUT).
  - TicketJsonStore.save(List<Ticket> tickets, Path file) — mapper.writeValue(file.toFile(), tickets).
  - List<Ticket> TicketJsonStore.load(Path file) — deserialize with a TypeReference<List<Ticket>>, so subtypes come back as their real classes, not generic maps.
  - Wrap Jackson's checked IOException/JsonProcessingException into your existing InvalidTicketDataException at the store's boundary — callers above this layer shouldn't need to know Jackson is involved.

  - Keep the T-04 pipe-delimited loader working as a one-time migration path: read the old format, write it out once as tickets.json, and use JSON as the format from then on.

Build T7 – JSON and Jackson Basics

Build T7 introduces JSON serialization and deserialization using the Jackson library.

The main objective is to understand how Java objects can be converted into JSON, stored in files, and reconstructed into Java objects later.

Your `TicketJsonStore` class implements this functionality for QueueDesk.

1. What is JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based data format used to store and exchange data.

It represents data using key-value pairs and arrays.

Example:

```
None
{
  "id": 101,
  "title": "Login page crashes",
  "priority": "HIGH",
  "status": "OPEN"
}
```

JSON is language-independent. Java, Python, JavaScript and other languages can read and generate JSON.

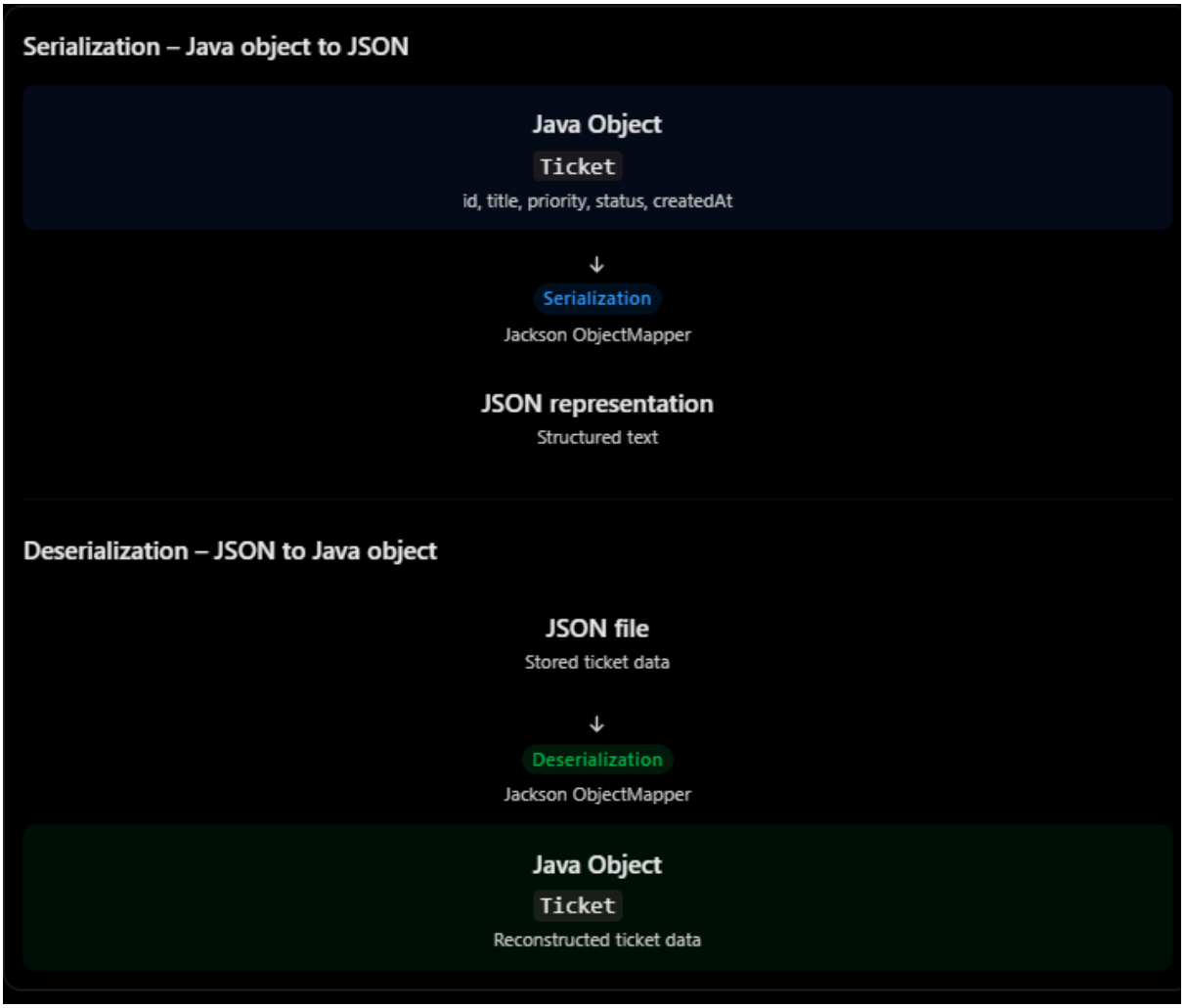
Why is JSON used in backend development?

- API communication between frontend and backend.
- Storing application data in files.
- Exchanging data between different services.
- Configuration files.
- Data transfer between systems.

QueueDesk example: Instead of storing tickets only in memory or a text file, we can save them in a structured JSON file and load them again when the application restarts.

2. Serialization vs Deserialization

These are the two most important concepts in Build T7.



Concept	Meaning	QueueDesk method
Serialization	Java object → JSON	`save()`
Deserialization	JSON → Java object	`load()`

Interview answer: Serialization converts Java objects into a format suitable for storage or transfer. Deserialization reconstructs Java objects from that stored or transferred data.

3. What is Jackson?

Jackson is a popular Java library used to process JSON data.

It provides functionality to:

- Convert Java objects into JSON.
- Convert JSON into Java objects.
- Handle collections and generic types.
- Configure JSON formatting.
- Support Java date and time classes through additional modules.
- Handle custom serialization and deserialization requirements.

Important Jackson classes used in your code

Class	Purpose
`ObjectMapper`	Main class for JSON conversion
`SerializationFeature`	Configures serialization behaviour
`JavaTimeModule`	Adds support for Java 8 date/time types
`TypeReference`	Preserves generic type information during deserialization and typed writing

4. Understanding ObjectMapper

Your code:

```
None
private final ObjectMapper mapper = new ObjectMapper()
```

```
    .registerModule(new JavaTimeModule())
    .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
    .enable(SerializationFeature.INDENT_OUTPUT);
```

Let's break it down.

What is ObjectMapper?

`ObjectMapper` is a class provided by Jackson.

```
None
ObjectMapper mapper = new ObjectMapper();
```

- `ObjectMapper` – class name.
- `mapper` – reference variable.
- `new ObjectMapper()` – creates an ObjectMapper instance.

The mapper is responsible for converting Java objects to JSON and JSON back to Java objects.

Why `private final`?

```
None
private final ObjectMapper mapper
```

- `private` – accessible only inside the class.
- `final` – the mapper reference cannot be reassigned after initialization.
- It is created once and reused by `save()` and `load()`.

A configured ObjectMapper can be reused across operations and is thread-safe after configuration is complete.

5. Method Chaining

Your ObjectMapper configuration uses method chaining.

```
None
ObjectMapper mapper = new ObjectMapper()
    .registerModule(new JavaTimeModule())
    .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
    .enable(SerializationFeature.INDENT_OUTPUT);
```

Method chaining means calling multiple methods one after another on the same object.

Instead of:

```
None
ObjectMapper mapper = new ObjectMapper();

mapper.registerModule(new JavaTimeModule());
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
mapper.enable(SerializationFeature.INDENT_OUTPUT);
```

We can write all the calls together.

Understand the three configuration methods

1. registerModule()

```
None
.registerModule(new JavaTimeModule())
```

- Registers an additional Jackson module.
- `JavaTimeModule` adds support for Java 8 date/time classes.
- Examples: `LocalDate`, `LocalDateTime`, `ZonedDateTime`.

Why do we need it in QueueDesk?

Your `Ticket` class contains:

```
None
private final LocalDateTime createdAt;
```

Jackson needs appropriate date/time support to serialize and deserialize this field.

2. disable()

```
None
.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
```

Disables writing date/time values as numeric timestamps.

Instead, dates are written in a human-readable ISO-8601 representation.

Example:

```
None
{
  "createdAt": "2026-11-10T15:30:00"
}
```

3. enable()

```
None
.enable(SerializationFeature.INDENT_OUTPUT)
```

Enables pretty-printing of JSON.

Without indentation:

```
None
{"id":101,"title":"Login page crashes"}
```

With indentation:

None

```
{
  "id": 101,
  "title": "Login page crashes"
}
```

It improves readability. It does not change the actual ticket data.

```
/*

Above performed is Method of chaining =

new ObjectMapper()

    .registerModule(new JavaTimeModule())

    .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)

    .enable(SerializationFeature.INDENT_OUTPUT);



rather than :

ObjectMapper mapper = new ObjectMapper();

mapper.registerModule(new JavaTimeModule());

mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);

mapper.enable(SerializationFeature.INDENT_OUTPUT);



ObjectMapper - class provided by jackson library and mapper = reference
variable , new ObjectMapper() = creates an ObjectMaper instance.

Jackson Methods -

1. registerModule() - is a method of ObjectMapper class that allows us
to register additional modules to customize the behavior of the
ObjectMapper. In this case, we are registering the JavaTimeModule to
handle Java 8 date and time types.
```

new JavaTimeModule() - creates a javaTimeModule object.

JavaTimeModule provides support for Java 8 date/time classes such as LocalDateTime.QueueDesk's ticket class contains createdAt field.Registering this module allows jackson to handle that field during json conversion.

2.disable() - is a method of ObjectMapper class that allows us to disable specific serialization features. In this case, we are disabling the WRITE_DATES_AS_TIMESTAMPS feature, which means that dates will be serialized in a more human-readable format (ISO-8601) instead of as timestamps.

```
.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
```

3.enable() - is a method of ObjectMapper class that allows us to enable specific serialization features. In this case, we are enabling the INDENT_OUTPUT feature, which means that the JSON output will be indented for better readability.

```
.enable(SerializationFeature.INDENT_OUTPUT)
```

This enables pretty-printing of JSON.

Without indentation:

```
{"id":101,"title":"Login page crashes"}
```

With indentation:

```
{
```

```
  "id": 101,

  "title": "Login page crashes"

}

*/
```

6. Understanding TypeReference

Your code uses:

```
None
new TypeReference<List<Ticket>>() {}
```

This is important because of Java Generics and type erasure.

What is TypeReference?

`TypeReference` is a Jackson class that helps specify the exact generic type expected during JSON processing.

Consider:

```
None
List<Ticket> tickets;
```

We want Jackson to understand that the JSON represents:

- A List.
- Containing Ticket objects.

Not just a generic List or a List of Strings.

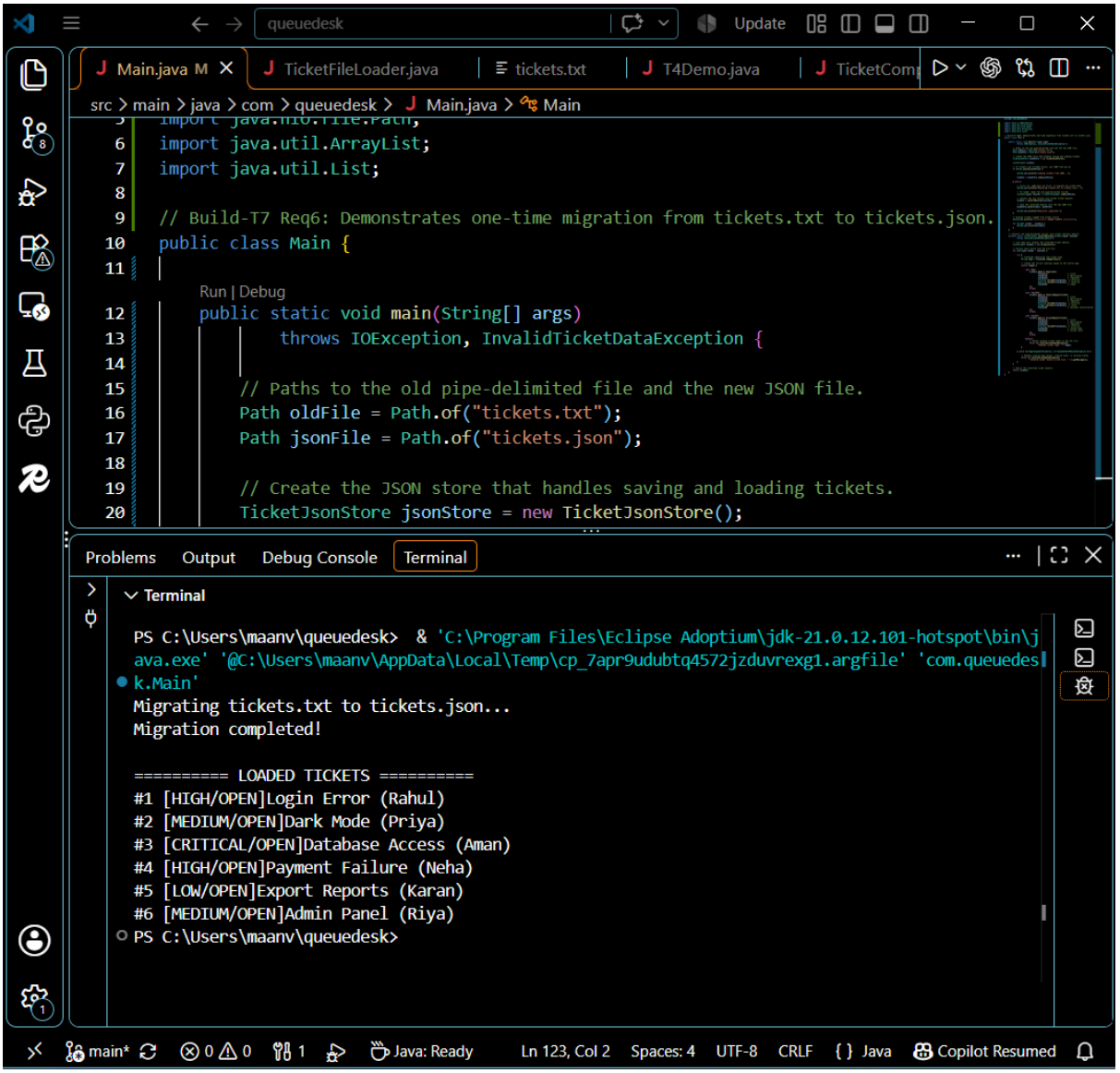
Why do we write `{}`?

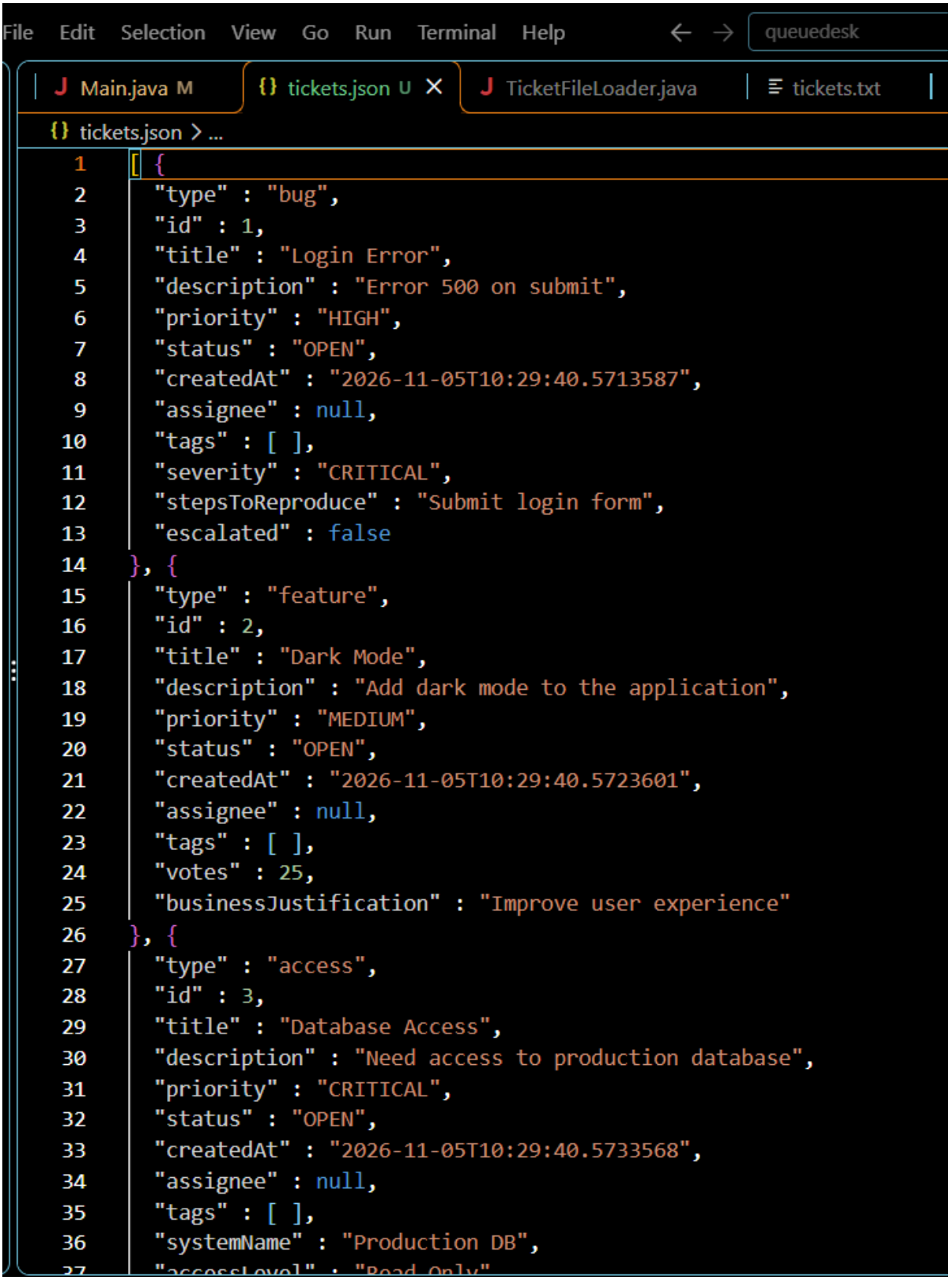
```
None
new TypeReference<List<Ticket>>() {}
```

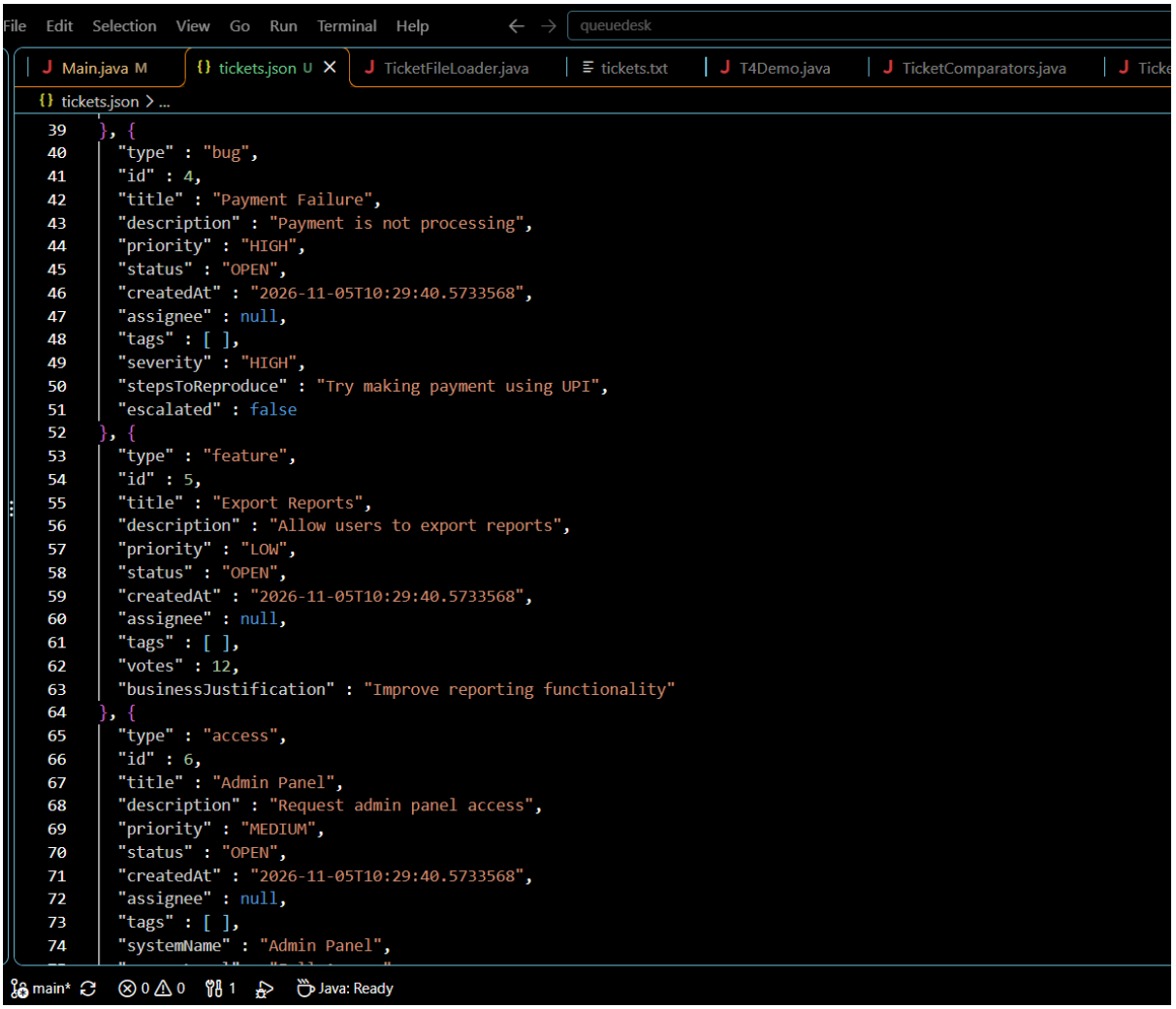
This creates an anonymous subclass of `TypeReference<List<Ticket>>`.

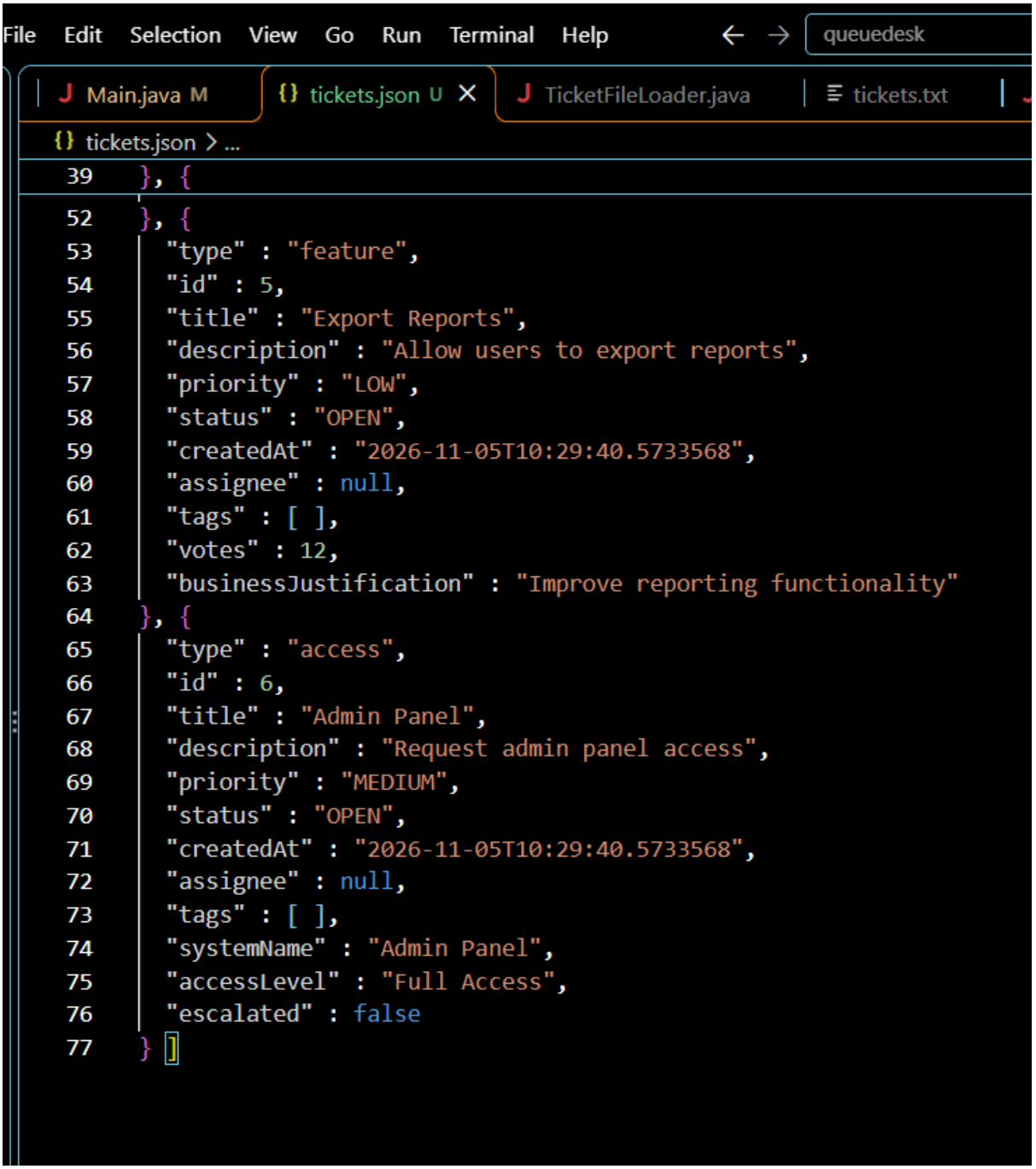
Jackson uses that subclass to retain the generic type information at runtime.

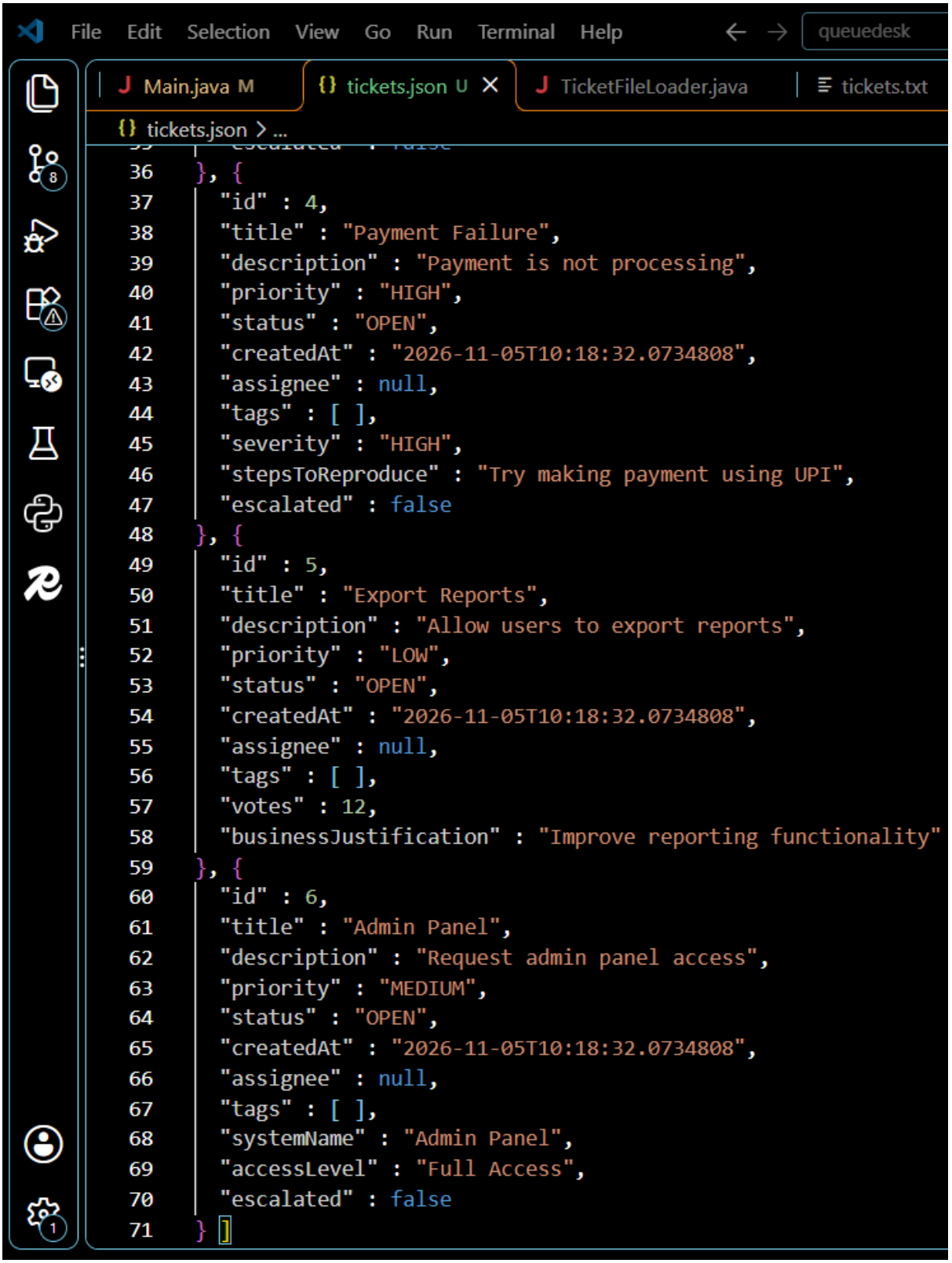
Interview answer: TypeReference is used when working with generic types such as `List<Ticket>`, because Java type erasure makes generic type information unavailable in the usual way at runtime.

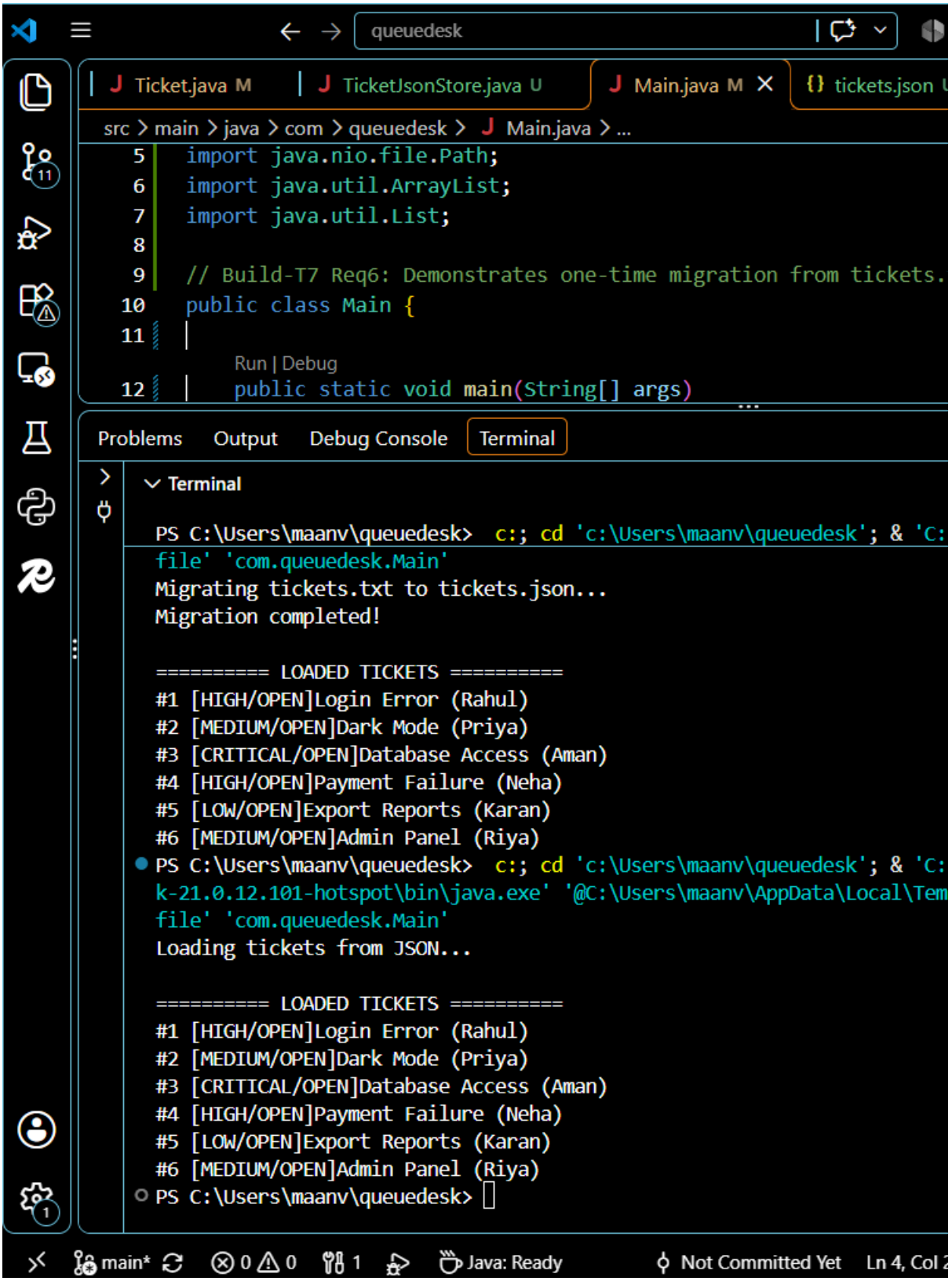



File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
[{
"type" : "bug",
"id" : 1,
"title" : "Login Error",
"description" : "Error 500 on submit",
"priority" : "HIGH",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5713587",
"assignee" : null,
"tags" : [ ],
"severity" : "CRITICAL",
"stepsToReproduce" : "Submit login form",
"escalated" : false
}, {
"type" : "feature",
"id" : 2,
"title" : "Dark Mode",
"description" : "Add dark mode to the application",
"priority" : "MEDIUM",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5723601",
"assignee" : null,
"tags" : [ ],
"votes" : 25,
"businessJustification" : "Improve user experience"
}, {
"type" : "access",
"id" : 3,
"title" : "Database Access",
"description" : "Need access to production database",
"priority" : "CRITICAL",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5733568",
"assignee" : null,
"tags" : [ ],
"systemName" : "Production DB",


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
T4Demo.java
TicketComparators.java
tickets.json > ...
39 }, {
40 "type" : "bug",
41 "id" : 4,
42 "title" : "Payment Failure",
43 "description" : "Payment is not processing",
44 "priority" : "HIGH",
45 "status" : "OPEN",
46 "createdAt" : "2026-11-05T10:29:40.5733568",
47 "assignee" : null,
48 "tags" : [ ],
49 "severity" : "HIGH",
50 "stepsToReproduce" : "Try making payment using UPI",
51 "escalated" : false
52 }, {
53 "type" : "feature",
54 "id" : 5,
55 "title" : "Export Reports",
56 "description" : "Allow users to export reports",
57 "priority" : "LOW",
58 "status" : "OPEN",
59 "createdAt" : "2026-11-05T10:29:40.5733568",
60 "assignee" : null,
61 "tags" : [ ],
62 "votes" : 12,
63 "businessJustification" : "Improve reporting functionality"
64 }, {
65 "type" : "access",
66 "id" : 6,
67 "title" : "Admin Panel",
68 "description" : "Request admin panel access",
69 "priority" : "MEDIUM",
70 "status" : "OPEN",
71 "createdAt" : "2026-11-05T10:29:40.5733568",
72 "assignee" : null,
73 "tags" : [ ],
74 "systemName" : "Admin Panel",
main*
0 0
1
Java: Ready


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
39 }, {
52 }, {
53 "type" : "feature",
54 "id" : 5,
55 "title" : "Export Reports",
56 "description" : "Allow users to export reports",
57 "priority" : "LOW",
58 "status" : "OPEN",
59 "createdAt" : "2026-11-05T10:29:40.5733568",
60 "assignee" : null,
61 "tags" : [ ],
62 "votes" : 12,
63 "businessJustification" : "Improve reporting functionality"
64 }, {
65 "type" : "access",
66 "id" : 6,
67 "title" : "Admin Panel",
68 "description" : "Request admin panel access",
69 "priority" : "MEDIUM",
70 "status" : "OPEN",
71 "createdAt" : "2026-11-05T10:29:40.5733568",
72 "assignee" : null,
73 "tags" : [ ],
74 "systemName" : "Admin Panel",
75 "accessLevel" : "Full Access",
76 "escalated" : false
77 }]


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
36 }, {
37 "id" : 4,
38 "title" : "Payment Failure",
39 "description" : "Payment is not processing",
40 "priority" : "HIGH",
41 "status" : "OPEN",
42 "createdAt" : "2026-11-05T10:18:32.0734808",
43 "assignee" : null,
44 "tags" : [ ],
45 "severity" : "HIGH",
46 "stepsToReproduce" : "Try making payment using UPI",
47 "escalated" : false
48 }, {
49 "id" : 5,
50 "title" : "Export Reports",
51 "description" : "Allow users to export reports",
52 "priority" : "LOW",
53 "status" : "OPEN",
54 "createdAt" : "2026-11-05T10:18:32.0734808",
55 "assignee" : null,
56 "tags" : [ ],
57 "votes" : 12,
58 "businessJustification" : "Improve reporting functionality"
59 }, {
60 "id" : 6,
61 "title" : "Admin Panel",
62 "description" : "Request admin panel access",
63 "priority" : "MEDIUM",
64 "status" : "OPEN",
65 "createdAt" : "2026-11-05T10:18:32.0734808",
66 "assignee" : null,
67 "tags" : [ ],
68 "systemName" : "Admin Panel",
69 "accessLevel" : "Full Access",
70 "escalated" : false
71 }]


queuedesk
Ticket.java M
TicketJsonStore.java U
Main.java M
tickets.json
src > main > java > com > queuedesk > Main.java > ...
5 import java.nio.file.Path;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 // Build-T7 Req6: Demonstrates one-time migration from tickets.
10 public class Main {
11
Run | Debug
12 public static void main(String[] args)
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:
file' 'com.queuedesk.Main'
Migrating tickets.txt to tickets.json...
Migration completed!
========== LOADED TICKETS ==========
#1 [HIGH/OPEN]Login Error (Rahul)
#2 [MEDIUM/OPEN]Dark Mode (Priya)
#3 [CRITICAL/OPEN]Database Access (Aman)
#4 [HIGH/OPEN]Payment Failure (Neha)
#5 [LOW/OPEN]Export Reports (Karan)
#6 [MEDIUM/OPEN]Admin Panel (Riya)
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:
k-21.0.12.101-hotspot\bin\java.exe' '@C:\Users\maanv\AppData\Local\Tem
file' 'com.queuedesk.Main'
Loading tickets from JSON...
========== LOADED TICKETS ==========
#1 [HIGH/OPEN]Login Error (Rahul)
#2 [MEDIUM/OPEN]Dark Mode (Priya)
#3 [CRITICAL/OPEN]Database Access (Aman)
#4 [HIGH/OPEN]Payment Failure (Neha)
#5 [LOW/OPEN]Export Reports (Karan)
#6 [MEDIUM/OPEN]Admin Panel (Riya)
PS C:\Users\maanv\queuedesk>
main*
0 0
1
Java: Ready
Not Committed Yet
Ln 4, Col 2

Unit Testing in Java – JUnit 5 + Mockito

1. Unit Testing

Unit testing is the process of testing individual units of code, such as methods or classes, independently to verify that they produce the expected results.

Why unit testing?

- Detecting bugs early.
- Verify expected behaviour.
- Prevent regression after code changes.
- Test invalid inputs and edge cases.
- Make code refactoring safer.

2. JUnit 5

JUnit 5 is a Java testing framework used to write and execute test cases.

Important annotations:

Annotation	Purpose
`@Test`	Marks a test method
`@BeforeEach`	Runs before every test
`@AfterEach`	Runs after every test
`@BeforeAll`	Runs once before all tests
`@AfterAll`	Runs once after all tests
`@Disabled`	Skips a test
`@DisplayName`	Gives a readable test name
`@ParameterizedTest`	Runs a test with multiple inputs

AAA testing pattern:

1. Arrange – Prepare test data and objects.
2. Act – Execute the method.
3. Assert – Verify the result.

Basic syntax:

```
None
@Test
void shouldAddTwoNumbers() {

    // Arrange
    int a = 4;
    int b = 6;

    // Act
    int result = a + b;

    // Assert
    assertEquals(10, result);
}
```

3. Important JUnit Assertions

Assertion	Purpose
`assertEquals()`	Checks equality
`assertNotEquals()`	Checks inequality
`assertTrue()`	Checks if condition is true
`assertFalse()`	Checks if condition is false
`assertNull()`	Checks if value is null
`assertNotNull()`	Checks if value is not null

`assertThrows()`	Checks if expected exception is thrown
`assertDoesNotThrow()`	Checks if no exception occurs
`assertAll()`	Groups multiple assertions

4. Mockito

Mockito is a Java mocking framework used with JUnit to create mock dependencies.

Mock: A fake object that replaces a real dependency during testing.

Why Mockito?

- Test a class independently of its dependencies.
- Avoid connecting to real databases or external APIs.
- Control the behaviour of dependencies.
- Verify interactions between classes.

Important annotations:

Annotation	Purpose
`@Mock`	Creates a mock object
`@InjectMocks`	Injects mocks into the class under test
`@ExtendWith(MockitoExtension.class)`	Integrates Mockito with JUnit 5

Important methods:

Method	Purpose
`when().thenReturn()`	Defines a return value
`when().thenThrow()`	Defines an exception

`verify()`	Checks if a method was called
`times(n)`	Checks number of calls
`never()`	Checks method was never called
`any()`	Matches any compatible argument
`ArgumentCaptor`	Captures method arguments

Mockito example:

```
None
@ExtendWith(MockitoExtension.class)
class TicketServiceTest {

    @Mock
    TicketRepository repository;

    @InjectMocks
    TicketService service;

    @Test
    void shouldReturnTicket() {

        Ticket ticket = createSampleTicket();

        // Arrange: define mock behaviour
        when(repository.findById(101))
            .thenReturn(ticket);

        // Act
        Ticket result = service.getTicket(101);

        // Assert
        assertEquals(ticket, result);

        // Verify interaction
```

```
        verify(repository).findById(101);
    }
}
```

5. Unit Testing vs Integration Testing

Unit Testing	Integration Testing
Tests individual units	Tests multiple components together
Often uses mocks	Often uses real dependencies
Usually faster	Usually slower
Example: TicketService with mocked Repository	Example: Save and load tickets from JSON file

QueueDesk application: TicketTest.java contains 14 JUnit 5 tests covering Ticket creation and validation, Repository operations, TicketAnalytics calculations, exceptions and sorting.

Remember: JUnit verifies expected behaviour; Mockito isolates dependencies and verifies interactions.

TicketTest.java

Your uploaded test file contains 14 JUnit 5 test cases. It uses JUnit assertions and directly creates objects instead of using Mockito.

Here is how your tests map to the concepts.

Test	What it verifies	JUnit concept
1	Ticket stores correct details	`assertEquals()`

2	Blank title is rejected	`assertThrows()`
3	Priority can be updated	`assertEquals()`
4	Tickets have unique IDs	`assertNotEquals()`
5	Repository stores tickets	`assertEquals()`
6	Repository finds ticket by ID	`assertEquals()`
7	Missing ticket ID throws exception	`assertThrows()`
8	Repository list is unmodifiable	`assertThrows()`
9	Counts tickets by priority	`assertEquals()`
10	Calculates average effort	`assertTrue()`
11	Calculates total effort	`assertTrue()`
12	Finds oldest OPEN ticket	`assertTrue()` + `assertEquals()`
13	Higher priority sorts first	`assertTrue()`

14	Comparator sorts by assignee	`assertTrue()`